

Demo 1 Test Notes

Monday, 02 March 2026 20:49

Components that were Implemented during Demo 1

- Reverse-Polarity diode
- L78C5 5V regulator
- LD1117V33 3.3V regulator
- LEDs
- Buttons
- Temperature Sensor

Software Implementation during Demo 1

- UART Student number
- LED flashing
- Button-toggling
- Temperature sensor
- Debouncing

Key Notes from Lecture 1 Slides

What is a decoupling/bypass capacitor

A small capacitor placed between the power line and GND to "clean up" power signal, i.e.:

- Smooths out fluctuation
- Eliminates high frequency noise
- Protect IC from unstable power

What is Dropout Voltage

The minimum difference between V_{in} and V_{out} at which the regulator can still maintain the correct output V

Reverse-Polarity Diode

Monday, 02 March 2026 21:12



1N4007
DIODE

✓ Purpose of the Reverse-Polarity Diode

A reverse-polarity diode is added so that **if the user accidentally connects the power supply backwards**, the circuit does **not** receive negative voltage.

✓ What happens without the diode

If the battery or power supply is connected incorrectly (e.g., +9V and GND swapped):

- negative voltage would be applied to the 7805 regulator
- ICs on the board would be reverse-biased
- electrolytic capacitors may be damaged
- components connected to the 5V rail may fail instantly

Many components are not protected against reverse voltage, so the damage is often permanent.

✓ What the diode actually does in your circuit

In your schematic, **D1 (1N4007)** is placed in series with the input. It acts as a **one-way valve**:

- If the power is connected correctly → diode conducts normally → the regulator receives power.
- If the power is connected backwards → the diode becomes reverse-biased → **no current flows** → the rest of the circuit sees **0V** instead of a negative voltage.

So instead of destroying your board, the circuit simply **does not turn on**—which is safe.

✓ Additional benefit

Series reverse-polarity diodes also protect against:

- accidentally plugging in a center-negative barrel jack
- connecting a bench supply incorrectly
- automotive wiring mistakes

L78C5 5V regulator

Monday, 02 March 2026 20:54



5V_Regula
tor_L78C5

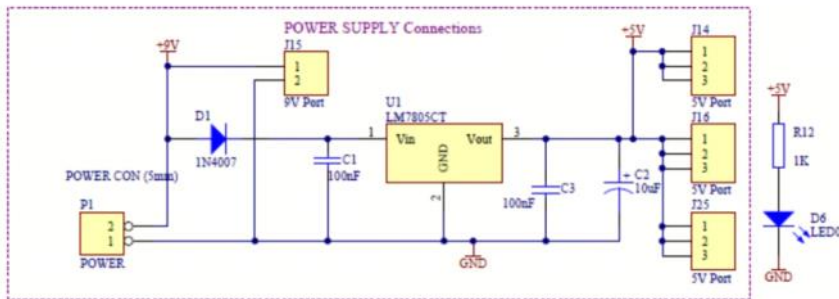


Figure 3: 5 V Power supply circuit diagram

What is the Purpose of C1 and C3? Why is C2 redundant?

● C1 (100 nF) — Input decoupling capacitor

Purpose:

Placed *as close as possible* to the 7805 input pin.

It provides **high-frequency bypassing** to ground, stabilizing the regulator by handling fast transient currents **before** they reach the 7805.

Why 100 nF?

Small ceramic caps have low ESR and handle high-frequency noise extremely well.

● C3 (100 nF) — Output decoupling capacitor

Purpose:

Also a **high-frequency bypass** cap, but on the output side.

This keeps the 7805 stable by preventing oscillation and giving it a clean, low-impedance load at high frequencies.

Why 100 nF?

Again: excellent for high-frequency stability.

● C2 (10 µF) — Bulk output capacitor

Purpose:

This is a larger capacitor meant for **low-frequency smoothing** and providing short bursts of extra current when the load changes suddenly (e.g., microcontroller waking up, LEDs switching on).

So far: **C2 is useful.**

! So why do people say C2 is “redundant”?

Because **the 7805 datasheet already requires only one output capacitor**, typically:

- 0.1 µF (for high-frequency stability)
- optionally 1 µF to 10 µF (for load transients)

But your circuit **already has C3 (100 nF)** for stability.

If your load is light and steady, the **extra electrolytic (C2 = 10 µF)** doesn't do much and can be seen as optional.

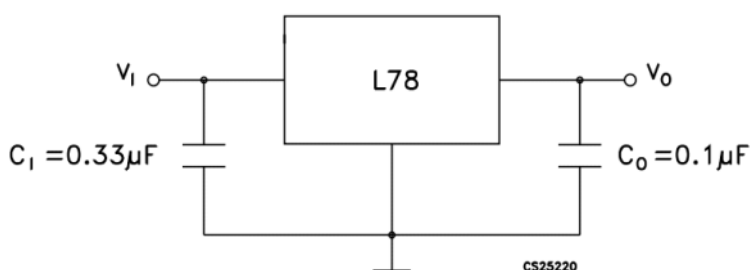
In other words:

✓ **C3 is required**

✓ **C2 is optional (nice to have, but not strictly required)**

C2 *isn't harmful* — it just isn't essential unless your 5V load draws sudden or pulsed currents.

Figure 4. Application circuits



✓ The datasheet calls for 0.33 µF on the input, but

Your **100 nF** ceramic C1 performs the same HF function *better* than the old 0.33 µF *tantalum/mylar* did.

Modern designs often replace the original 0.33 µF with **100 nF** + bulk capacitance upstream.

✓ **C3 (100 nF) matches the datasheet's 0.1 µF required output bypass**

✓ **C2 (10 µF) is optional bulk storage**

It improves transient performance but is **not required for regulator stability**, which is why many people call it “redundant”.

Is a Heat-Sink Necessary For the Regulator

It depends on how much power your 7805 is dissipating.

You only need a heat sink if the regulator is dissipating **more heat than the TO-220 package can safely handle without one.**

Below is the clear way to decide.

✓ 1. The key formula: how much heat does your 7805 produce?

Linear regulators turn the voltage drop into heat: $P_{\text{dissipated}} = (V_{\text{in}} - V_{\text{out}}) \times I_{\text{load}}$

Example from ProtoSupplies:

Using 9 V → 5 V at 1 A produces **4 W** of heat.

A 7805 **definitely needs a heatsink** at that level.

✓ 2. How much can a 7805 dissipate without a heatsink?

Typical safe dissipation (TO-220 package):

- **~1–1.25 W without a heatsink** (depending on airflow and ambient temperature).

This is confirmed by real-world measurements and forum engineering discussions, which note that around **1.25 W** is the upper limit before needing a sink.

If your calculated power is **below ~1 W**, you are generally safe without a sink.

✓ 3. Examples

Case A: 9 V input, 5 V output, 200 mA load

$$P = (9 - 5) \times 0.2 = 0.8 \text{ W}$$

→ No heatsink needed; 0.8 W is within the 1 W limit.

Case B: 9 V input, 5 V output, 500 mA load

$$P = 4 \times 0.5 = 2 \text{ W}$$

→ Heatsink required (2 W > 1 W limit).

Case C: 12 V input, 5 V output, 300 mA load

$$P = 7 \times 0.3 = 2.1 \text{ W}$$

→ Heatsink definitely required.

✓ 4. When you DO need a heatsink

You need a heatsink if **any** of these are true:

- $(V_{\text{in}} - V_{\text{out}}) \times I_{\text{load}} > 1 \text{ W}$
- The regulator becomes too hot to touch (above ~70°C case)
- You are using it inside a closed plastic enclosure
- You draw $\geq 300 \text{ mA}$ from 9–12 V
- You draw $> 100 \text{ mA}$ from 12–15 V

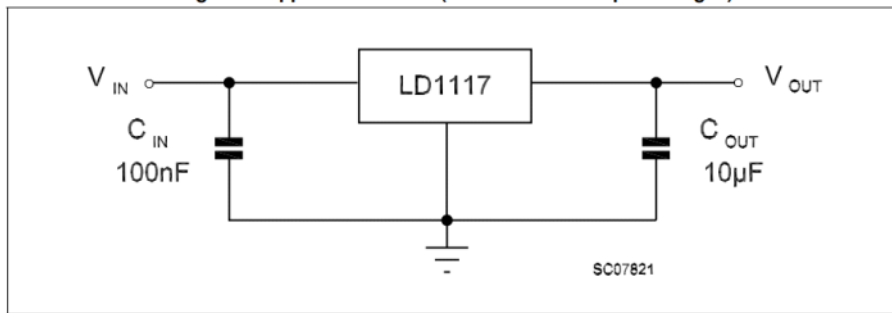
👉 If your load is:

- 100–250 mA from a 9 V supply → Probably not
- 300 mA or more from 9–12 V → Yes
- 500 mA or more → Definitely yes
- >1 A → Use a switching regulator instead; a heatsink alone won't be enough.

LD1117V33 3.3V regulator

Monday, 02 March 2026 21:17

Figure 4. Application circuit (for other fixed output voltages)



Notes on the 3.3 V Regulator (LD1117V33)

1. Purpose of the LD1117V33

The LD1117V33 is a **low-dropout (LDO) linear voltage regulator** designed to convert a higher input voltage down to a **regulated 3.3 V** output. It can supply **up to 800 mA** of load current. It belongs to the LD1117 family, which offers several fixed-output versions.

2. Key Features

- **Low dropout voltage:** Typically **1.0 V** at 100 mA and up to **1.2 V** at 800 mA.
- **Fixed 3.3 V output** option available (LD1117#33).
- **Output current up to 800 mA.**
- **Internal current limit** and **thermal shutdown** protection.
- **Requires only one output capacitor** for stability: **10 µF** minimum.
- Tight output tolerance: **±1%** at 25 °C.

3. Input Voltage Requirements

- Maximum allowable input voltage: **15 V**.
- Minimum input for regulated 3.3 V output: $V_{IN(min)} \approx 3.3 \text{ V} + \text{dropout} (\approx 1.1 \text{ V}) \approx 4.4 \text{ V}$
- Recommended to keep **VIN between 4.75 V** and 10V for best performance (datasheet test conditions).

4. Dropout Voltage

From the 3.3 V version's performance table:

Load current	Typical dropout	Max dropout
100 mA	1.0 V	1.1 V
500 mA	1.05 V	1.15 V
800 mA	1.10 V	1.2 V

Because of the significant dropout voltage, the LD1117 cannot regulate properly if the input is too close to 3.3 V.

5. Required Capacitors and Stability

Input capacitor

- The datasheet application circuit shows the input capacitor as optional, but usually included if the regulator is far from the supply or if the supply is noisy.

Output capacitor (mandatory)

- Minimum **10 µF** capacitor required on the output for stable operation.
- Must have **low ESR** (common electrolytic or tantalum preferred).

This is different from the 7805, which needs **100 nF** + 0.1–10 µF, while the LD1117 specifically depends on the $\approx 10 \mu\text{F}$ capacitor for loop stability.

6. Power Dissipation and Thermal Considerations

Like all linear regulators, the LD1117 dissipates power as heat:

$$P = (V_{IN} - V_{OUT}) \times I_{OUT}$$

The TO-220 package thermal resistance values:

- **RθJA = 50 °C/W** (junction-to-ambient)
- **RθJC = 5 °C/W** (junction-to-case)

Because RθJA is similar to the 7805's, the thermal guidelines are the same:

Heatsink needed if:

- Dissipation > ~1 W without heatsink
- Load > ~300 mA from 5 V or 9 V input
- Regulator runs hot to the touch

For example, with 9 V input:

$$P = (9 - 3.3) \times 0.3 \text{ A} = 1.71 \text{ W} \Rightarrow \text{Heatsink required}$$

7. Protection Features

The regulator includes:

- **Internal current limiting** to prevent overload damage.
- **Thermal shutdown**, which turns off the device if the die reaches excessive temperature.

These features prevent permanent failure but **do not** replace the need for proper heatsinking.

8. Comparison to 5 V (7805) Regulator

Feature	7805	LD1117V33
Dropout voltage	~2 V	~1.1 V
Max current	1–1.5 A	800 mA
Required caps	0.33 μ F input, 0.1 μ F output	10 μF output mandatory
Max VIN	35 V	15 V
Type	Standard linear regulator	Low-dropout linear regulator
Heat behaviour	Very similar	Very similar

The LD1117 is better for battery-powered or low-voltage input systems because of its lower dropout.

Light-Emitting Diodes

Monday, 02 March 2026 21:41



3_3V

Regulator...

1. Demo 1 System Requirements for LEDs

According to the system specifications for Demo 1, you need to implement the following LED hardware:

- **Power Indicator (D6):** This LED must turn on to indicate that the system's 5V regulated voltage is present.
- **Status/Alarm Indicators (D2-D5):** The system uses a set of LEDs to act as visual alarm indicators for different warnings.
 - D2 acts as the proximity warning indicator.
 - D3 indicates unsafe driving and detected impacts.
 - D4 indicates low-light conditions.
 - D5 indicates uncomfortable temperatures.
- **Flashing Specifications:** The flashing timing for these status-indicating LEDs must be exactly 500 ms ON and 500 ms OFF.
- **Testing:** During Demo 1, the Test Station (TS) will visually monitor the power status LED (D6) and the status-indicating LEDs (D2-D5) to ensure they operate correctly on power-up.

2. Relevant Electrical Characteristics

To properly implement the Super Orange Red LED (NOR28L32) into your circuit, you need its electrical and optical characteristics at an ambient temperature of 25°C:

- **Forward Voltage (V_F):** The typical forward voltage is 2.0 V, with a minimum of 1.9 V and a maximum of 2.6 V.
- **Forward Current (I_F):** The recommended operating current is 20 mA.
- **Peak Forward Current:** It can handle a peak of 100 mA, tested at a 1/10 duty cycle at 1KHz.
- **Luminous Intensity (I_V):** The typical luminous intensity is 2500 mcd, with a maximum of 3000 mcd.
- **Reverse Voltage (V_R):** The absolute maximum reverse voltage the LED can withstand is 5 V.
- **Operating Temperature:** The LED is rated to operate safely between -40°C and 85°C.

3. Purpose of the Current-Limiting Resistor

- Because an LED is a current-operated device, it requires a current-limiting resistor to be incorporated into the drive circuit.
- This resistor drops the excess voltage from the supply and limits the current flowing through the LED to a safe level (such as the recommended 20 mA).
- The manufacturer specifically recommends using a current-limiting resistor to ensure intensity uniformity when connecting LEDs in a circuit application.
- D6 is connected to the L78C5 (5V) regulator, $\therefore V_{supply} = 5V$
- From datasheet: $V_{F(TYP)} = 2V$, $I_{LED} = 0.02A$
 - Normally use TYP values for calculations
 - $\therefore R = \frac{5 - 2}{0.02} = 150\Omega$
- D2, D3, D4, D5 is connected to a GPIO pin with a voltage rating 3.3V (pg. 38 from STM32f411RE datasheet)
 - $\therefore R = \frac{3.3 - 2}{0.02} = 65\Omega \rightarrow 100\Omega$ closest approximate

Buttons

Monday, 02 March 2026 22:10



Button_Tac
tile_Switch

1. Demo 1 System Requirements for Buttons (SS8)

According to the system specifications for Demo 1, your tactile push-buttons (SS8) will be rigorously tested by the Test Station (TS):

- **Response Time:** The maximum allowable response time after any button push is exactly 500 ms. You must ensure your system registers and reacts to the input within this window.

2. Relevant Electrical & Mechanical Characteristics

To properly implement the tactile push-buttons, you should be familiar with the following component specifications from the manufacturer:

- **Maximum Rating:** The buttons can handle a maximum of DC 12 V and 50 mA.
- **Contact Resistance:** When closed, the contact resistance is 50 m Ω Max.
- **Insulation Resistance:** When open, the insulation resistance is 100 M Ω Min (tested at DC 100V).
- **Bounce Time:** The mechanical bounce when testing at "ON" and "OFF" is a maximum of 5 ms.
- **Actuating Force:** It takes a force of 250 ± 50 gf to trigger the button.
- **Travel Distance:** The physical travel of the button stem is 0.3 ± 0.1 mm.
- **Operating Temperature:** The switches operate safely between -20°C and 70°C.

3. Hardware Implementation: Resistors and Debouncing

When integrating these buttons into your STM32 microcontroller circuit, you must account for two major hardware factors:

- **Pull-up / Pull-down Resistors:** When a tactile switch is unpressed (open), the microcontroller's GPIO pin is physically disconnected and left "floating." This will cause unpredictable logic states (picking up random electrical noise). To fix this, you must implement a **pull-up resistor** (tying the pin to your 3.3V logic high) or a **pull-down resistor** (tying the pin to ground). This ensures the pin sits at a known state until the button is pressed to pull it to the opposite state. *(Note: The STM32 has internal pull-up/pull-down resistors that you can enable in software, but you must know how to implement them in hardware if asked).*
- **Hardware Debouncing:** Because the spec sheet explicitly states the button has a bounce time of up to 5 ms, pressing it once will cause the electrical signal to rapidly spike between HIGH and LOW for a few milliseconds before settling. If unaccounted for, the micro controller will register a single push as multiple rapid pushes. While you can handle this in software (by adding a >5ms delay after an initial trigger), you can also implement **hardware debouncing** using a simple RC (Resistor-Capacitor) low-pass filter circuit to smooth out the fluctuating signal.

LMT01 Temperature Sensor

Monday, 02 March 2026 22:21



LMT01
Temperat...

1. Demo 1 System Requirements for the Temperature Sensor

According to the system specifications for Demo 1:

- **Testing & Verification:** The Test Station (TS) will monitor and test the output of the digital temperature sensor via UART.
- **UART Communication:** The measured temperature data must be transmitted using the clearly specified Demonstration 1 UART message format.

2. Relevant Electrical Characteristics (LMT01)

To successfully implement the LMT01 temperature sensor in your hardware, you must understand its unique operating parameters:

- **Sensor Type:** The LMT01 is a highly accurate, 2-pin digital output temperature sensor.
- **Current Loop Interface:** It utilizes a pulse count current loop interface that is easily read by a processor. The number of output pulses it transmits is proportional to the temperature, offering a 0.0625°C resolution.
- **Accuracy:** It maintains a maximum accuracy of $\pm 0.5^\circ\text{C}$ between -20°C and 90°C .
- **Base Current:** During operation, it draws a conversion current of $34\mu\text{A}$.
- **Frequency & Period:** The communication frequency of the sensor is 88 kHz. The continuous conversion plus data-transmission period is 100 ms.
- **Supply:** The sensor features a floating 2-V to 5.5-V (VP-VN) supply operation with integrated EMI immunity.

3. Hardware Implementation Details

- **Microcontroller Interface:** The LMT01's pulse count interface can be connected to the microcontroller using either a GPIO pin or a Comparator (COMP) input.

OUTPUT CURRENT					
I_{OL}	Output current variation	Low level	28	34	40 μA
I_{OH}		High level	112.5	125	143 μA
High-to-Low level output current ratio			3.1	3.7	4.5

- **Current Loop to Voltage Conversion:** The LMT01 temperature sensor transmits its data using a pulse count *current loop* interface, meaning it communicates by modulating the amount of current it draws. However, the microcontroller (STM32) reads *voltage* changes on its GPIO or Comparator (COMP) pins.
- **Applying Ohm's Law:** By placing a pull-down resistor between the sensor's VN terminal and ground, the current flowing out of the sensor is forced to pass through this resistor. According to Ohm's Law, this current creates a voltage drop across the resistor.
 - For $(v_o)_{STM} = 0.99\text{V}$: $R_H = \frac{0.99}{34 \times 10^{-6}} = 29.12\text{k}\Omega$
 - For $(v_o)_{STM} = 2.31\text{V}$: $R_H = \frac{2.31}{125 \times 10^{-6}} = 18.48\text{k}\Omega$
- **Creating Readable Pulses:** When the LMT01 pulses its current high and low to transmit temperature data, the pull-down resistor converts those fluctuating currents into corresponding high and low *voltage* pulses. This allows the microcontroller to easily read and count the pulses to determine the temperature.
- **Timing Constraints (Lecture Advice):** The lecture slides explicitly highlight the strict timing windows you must account for when reading the pulses:
 - There is a **50 ms** window between the start and the end of the data transmission.
 - There is a **104 ms** interval between the start of one data transmission until the start of the next data transmission.
- **Data Handling:** You should calculate an accurate mean and avoid anomalies when processing the readings.

UART Student number

Monday, 02 March 2026 22:41

```
void SendStudent_Number(void)
{
    HAL_Delay(350);
    char uart_buffer[20];
    const char studentNumber[] = "*24989142#\n";
    sprintf(uart_buffer, "%s", studentNumber);
    HAL_UART_Transmit(&huart2, (uint8_t*)studentNumber, strlen(studentNumber), 500);
}
```

1. UART Communication: Analysis and Implementation

The core software task for Demo 1 is managing UART communication on the USART2 peripheral. This involves sending a startup message and processing an incoming command.

1.1. UART Initialization (MX_USART2_UART_Init)

The USART2 peripheral must be configured with parameters that match the Test Station's expectations. The initialization code sets up the following:

```
huart2.Instance = USART2;
huart2.Init.BaudRate = 57600;
huart2.Init.WordLength = UART_WORDLENGTH_9B; // 8 data bits + 1 parity bit
huart2.Init.StopBits = UART_STOPBITS_1;
huart2.Init.Parity = UART_PARITY_EVEN;
huart2.Init.Mode = UART_MODE_TX_RX;
huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
huart2.Init.OverSampling = UART_OVERSAMPLING_16;
huart2.Init.OneBitSampling = UART_ONE_BIT_SAMPLE_DISABLE;
huart2.Init.ClockPrescaler = UART_PRESCALER_DIV1;
huart2.AdvancedInit.AdvFeatureInit = UART_ADVFEATURE_NO_INIT;
```

- **Insight:** This configuration is critical. Any terminal program used for testing must be set to **57600 baud, 8 data bits, Even parity, and 1 stop bit**. The `UART_WORDLENGTH_9B` setting is how the STM32 HAL accommodates the parity bit alongside the data bits

1.2. Transmitting the Student Number on Startup (SS2)

The `SendStudent_Number` function is called once after initialization to meet the first requirement of Demo 1.

```
void SendStudent_Number(void) {
    HAL_Delay(150);
    char uart_buffer[20];
    const char studentNumber[] = "*24989142#\n";
    sprintf(uart_buffer, "%s", studentNumber);
    HAL_UART_Transmit(&huart2, (uint8_t*)studentNumber, strlen(studentNumber), 500);
}
```

- **Transmission Time Calculation:**
 - Each character frame consists of 1 start bit + 8 data bits + 1 parity bit + 1 stop bit = 10 bits
 - The string `*24989142#\n` is 11 characters long
 - Total bits to transmit: $11 \text{ characters} \times 10 \frac{\text{bits}}{\text{character}} = 110 \text{ bits}$
 - Time to transmit at 57600 bps: $\frac{110 \text{ bits}}{57600 \text{ bits/sec}} \approx 1.91 \text{ ms}$. This is extremely fast and well within the 500 ms timeout

LED flashing

Monday, 02 March 2026 22:42

```
void UpdateLEDs(void)
{
    uint32_t now = HAL_GetTick();

    // 2 Hz flash = 500ms ON, 500ms OFF
    if ((now - lastFlashTime) >= 500)
    {
        lastFlashTime = now;
        flashToggle ^= 1;

        // D3 = PB4, flashes when state is ON
        if (led_d3_state)
            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, flashToggle ? GPIO_PIN_SET : GPIO_PIN_RESET);
        else
            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, GPIO_PIN_RESET);

        // D5 = PC4, flashes when state is ON
        if (led_d5_state)
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_4, flashToggle ? GPIO_PIN_SET : GPIO_PIN_RESET);
        else
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_4, GPIO_PIN_RESET);
    }

    // D2 = PB5, static ON/OFF
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, led_d2_state ? GPIO_PIN_SET : GPIO_PIN_RESET);

    // D4 = PA10, static ON/OFF
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, led_d4_state ? GPIO_PIN_SET : GPIO_PIN_RESET);
}
```

Button-pressing and Debouncing

Monday, 02 March 2026 22:44

```
void S1_Button(void)
{
    static uint8_t state = 0;
    static uint32_t press_time = 0;

    uint8_t current = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0);
    if (current == 1 && state == 0)
    {
        // Button just pressed
        press_time = HAL_GetTick();
        state = 1;
    }
    if (current == 0 && state == 1)
    {
        // Button released
        uint32_t duration = HAL_GetTick() - press_time;
        if (duration > 100) // must be held 100ms
        {
            led_d2_state = !led_d2_state;
        }
        state = 0;
    }
}
```

```
void S3_Button(void)
{
    static uint8_t state = 0;
    static uint32_t press_time = 0;

    uint8_t current = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_7);

    if (current == 1 && state == 0)
    {
        press_time = HAL_GetTick();
        state = 1;
    }

    if (current == 0 && state == 1)
    {
        uint32_t duration = HAL_GetTick() - press_time;

        if (duration > 100)
        {
            char msg[50];

            if (temp_ready)
                sprintf(msg, "@%.1f&\r\n", current_temp);
            else
                sprintf(msg, "Reading sensor...\r\n");

            HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), 100);
        }

        state = 0;
    }
}
```

Temperature Sensor

Monday, 02 March 2026 22:45

```
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    if ((htim->Instance == TIM3) && (htim->Channel == HAL_TIM_ACTIVE_CHANNEL_2))
    {
        uint32_t capture = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_2);

        uint32_t diff;
        if (capture >= last_capture_time)
            diff = capture - last_capture_time;
        else
            diff = (0xFFFF - last_capture_time + capture + 1);

        last_capture_time = capture;

        if (diff != 0)
        {
            float timer_clock = HAL_RCC_GetPCLK1Freq() * 2.0f;
            float frequency = timer_clock / diff;

            // ☒ Correct conversion
            float temperature = (frequency / 4096.0f) - 50.0f;

            // ☒ Filter AFTER conversion
            current_temp = CalibrateTemperature(temperature);

            temp_ready = 1;
        }
    }
}
```

```
float CalibrateTemperature(float new_temp)
{
    temp_buffer[temp_index++] = new_temp;

    if (temp_index >= TEMP_BUFFER_SIZE)
    {
        temp_index = 0;
        buffer_full = 1;
    }

    if (!buffer_full) return new_temp;

    // Copy buffer
    float temp_copy[5];
    memcpy(temp_copy, temp_buffer, sizeof(temp_buffer));

    // Sort (simple bubble sort)
    for (int i = 0; i < 4; i++)
        for (int j = i + 1; j < 5; j++)
            if (temp_copy[i] > temp_copy[j])
            {
                float t = temp_copy[i];
                temp_copy[i] = temp_copy[j];
                temp_copy[j] = t;
            }

    // Average middle 3 values
    return (temp_copy[1] + temp_copy[2] + temp_copy[3]) / 3.0f;
}
```

Demo 1 Software Implementation

Monday, 02 March 2026 22:24

```
/* USER CODE BEGIN Header */
/**
 * *****
 * @file           : main.c
 * @brief          : Main program body
 * *****
 * @attention
 *
 * Copyright (c) 2026 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
 * *****
 */
/* USER CODE END Header */
/* Includes ----- */
#include "main.h"

/* Private includes ----- */
/* USER CODE BEGIN Includes */
#include <string.h>
#include <stdio.h>

/* USER CODE END Includes */

/* Private typedef ----- */
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ----- */
/* USER CODE BEGIN PD */
#define BUTTON_DEBOUNCE_TIME_MS 200
#define TEMP_BUFFER_SIZE 5

/* USER CODE END PD */

/* Private macro ----- */
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ----- */
TIM_HandleTypeDef htim3;

UART_HandleTypeDef huart2;

/* USER CODE BEGIN PV */
// LED state tracking
static uint8_t led_d2_state = 1; // D2 starts ON (continuously)
uint8_t led_d3_state = 1; // D3 starts flashing
static uint8_t led_d4_state = 1; // D4 starts ON (continuously)
uint8_t led_d5_state = 1; // D5 starts flashing

// Flash timing
uint32_t lastFlashTime = 0;
```

```

uint8_t flashToggle = 0;

// Debounce trackers for each specific button
uint32_t LastTick_S1_PA0 = 0; // S1
uint32_t LastTick_S2_PB8 = 0; // S2
uint32_t LastTick_S3_PA7 = 0; // S3
uint32_t LastTick_S4_PB6 = 0; // S4
uint32_t LastTick_S5_PB9 = 0; // S5

// Temperature sensor tracking
volatile uint32_t pulse_count = 0;
volatile uint32_t last_capture_time = 0;
volatile float current_temp = -50.0f;
volatile uint8_t temp_ready = 0;

// Temperature sensor calibration
float temp_buffer[TEMP_BUFFER_SIZE] = {0};
uint8_t temp_index = 0;
uint8_t buffer_full = 0;

/* USER CODE END PV */

/* Private function prototypes -----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_USART2_UART_Init(void);
static void MX_TIM3_Init(void);
/* USER CODE BEGIN PFP */
float CalibrateTemperature(float new_temp);

/* USER CODE END PFP */

/* Private user code -----*/
/* USER CODE BEGIN 0 */
void SendStudent_Number(void)
{
    HAL_Delay(350);
    char uart_buffer[20];
    const char studentNumber[] = "*24989142#\n";
    sprintf(uart_buffer, "%s", studentNumber);
    HAL_UART_Transmit(&huart2, (uint8_t*)studentNumber, strlen(studentNumber), 500);
}

void UpdateLEDs(void)
{
    uint32_t now = HAL_GetTick();

    // 2 Hz flash = 500ms ON, 500ms OFF
    if ((now - lastFlashTime) >= 500)
    {
        lastFlashTime = now;
        flashToggle ^= 1;

        // D3 = PB4, flashes when state is ON
        if (led_d3_state)
            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, flashToggle ? GPIO_PIN_SET : GPIO_PIN_RESET);
        else
            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, GPIO_PIN_RESET);

        // D5 = PC4, flashes when state is ON
        if (led_d5_state)
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_4, flashToggle ? GPIO_PIN_SET : GPIO_PIN_RESET);
        else
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_4, GPIO_PIN_RESET);
    }
}

```

```

}

// D2 = PB5, static ON/OFF
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, led_d2_state ? GPIO_PIN_SET : GPIO_PIN_RESET);

// D4 = PA10, static ON/OFF
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, led_d4_state ? GPIO_PIN_SET : GPIO_PIN_RESET);
}

void S1_Button(void)
{
    static uint8_t state = 0;
    static uint32_t press_time = 0;

    uint8_t current = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0);
    if (current == 1 && state == 0)
    {
        // Button just pressed
        press_time = HAL_GetTick();
        state = 1;
    }
    if (current == 0 && state == 1)
    {
        // Button released
        uint32_t duration = HAL_GetTick() - press_time;
        if (duration > 100) // must be held 100ms
        {
            led_d2_state = !led_d2_state;
        }
        state = 0;
    }
}

void S2_Button(void)
{
    static uint8_t state = 0;
    static uint32_t press_time = 0;

    uint8_t current = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_8);

    if (current == 1 && state == 0)
    {
        // Button just pressed
        press_time = HAL_GetTick();
        state = 1;
    }
    if (current == 0 && state == 1)
    {
        // Button released
        uint32_t duration = HAL_GetTick() - press_time;
        if (duration > 100) // must be held 100ms
        {
            led_d3_state = !led_d3_state;
        }
        state = 0;
    }
}

void S3_Button(void)
{
    static uint8_t state = 0;
    static uint32_t press_time = 0;

    uint8_t current = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_7);

```



```

    if (current == 1 && state == 0)
    {
        press_time = HAL_GetTick();
        state = 1;
    }

    if (current == 0 && state == 1)
    {
        uint32_t duration = HAL_GetTick() - press_time;

        if (duration > 100)
        {
            char msg[50];

            if (temp_ready)
                sprintf(msg, "@%.1f&\r\n", current_temp);
            else
                sprintf(msg, "Reading sensor...\r\n");

            HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), 100);
        }

        state = 0;
    }
}

void S4_Button(void)
{
    static uint8_t state = 0;
    static uint32_t press_time = 0;

    uint8_t current = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_6);

    if (current == 1 && state == 0)
    {
        // Button just pressed
        press_time = HAL_GetTick();
        state = 1;
    }
    if (current == 0 && state == 1)
    {
        // Button released
        uint32_t duration = HAL_GetTick() - press_time;
        if (duration > 100) // must be held 100ms
        {
            led_d4_state = !led_d4_state;
        }
        state = 0;
    }
}

void S5_Button(void)
{
    static uint8_t state = 0;
    static uint32_t press_time = 0;

    uint8_t current = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_9);

    if (current == 1 && state == 0)
    {
        // Button just pressed
        press_time = HAL_GetTick();
        state = 1;
    }
}

```

```

    }
    if (current == 0 && state == 1)
    {
        // Button released
        uint32_t duration = HAL_GetTick() - press_time;
        if (duration > 100)    // must be held 100ms
        {
            led_d5_state = !led_d5_state;
        }
        state = 0;
    }
}

/* USER CODE END 0 */

/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{
    /* USER CODE BEGIN 1 */

    /* USER CODE END 1 */

    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    HAL_Init();

    /* USER CODE BEGIN Init */

    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* USER CODE BEGIN SysInit */

    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_USART2_UART_Init();
    MX_TIM3_Init();
    /* USER CODE BEGIN 2 */
    SendStudent_Number();

    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_SET);    // D2 ON
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, GPIO_PIN_SET);    // D3 starts ON
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_SET);    // D4 ON
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_4, GPIO_PIN_SET);    // D5 starts ON

    flashToggle = 1;
    lastFlashTime = HAL_GetTick();

    // Start TIM3 input capture in interrupt mode
    HAL_TIM_IC_Start_IT(&htim3, TIM_CHANNEL_2);

    /* USER CODE END 2 */

    /* Infinite loop */

```

```

/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    UpdateLEDs();
    S1_Button();
    S2_Button();
    S3_Button(); // sends current_temperature on press
    S4_Button();
    S5_Button();
    }
/* USER CODE END 3 */
}

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /** Configure the main internal regulator output voltage
    */
    __HAL_RCC_PWR_CLK_ENABLE();
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);

    /** Initializes the RCC Oscillators according to the specified parameters
    * in the RCC_OscInitTypeDef structure.
    */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    RCC_OscInitStruct.HSIState = RCC_HSI_ON;
    RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
    RCC_OscInitStruct.PLL.PLLM = 16;
    RCC_OscInitStruct.PLL.PLLN = 336;
    RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV4;
    RCC_OscInitStruct.PLL.PLLQ = 4;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }

    /** Initializes the CPU, AHB and APB buses clocks
    */
    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                                   |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
    {
        Error_Handler();
    }
}

/**
 * @brief TIM3 Initialization Function

```

```

* @param None
* @retval None
*/
static void MX_TIM3_Init(void)
{

/* USER CODE BEGIN TIM3_Init 0 */

/* USER CODE END TIM3_Init 0 */

TIM_MasterConfigTypeDef sMasterConfig = {0};
TIM_IC_InitTypeDef sConfigIC = {0};

/* USER CODE BEGIN TIM3_Init 1 */

/* USER CODE END TIM3_Init 1 */
htim3.Instance = TIM3;
htim3.Init.Prescaler = 0;
htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
htim3.Init.Period = 65535;
htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
htim3.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
if (HAL_TIM_IC_Init(&htim3) != HAL_OK)
{
    Error_Handler();
}
sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
if (HAL_TIMEx_MasterConfigSynchronization(&htim3, &sMasterConfig) != HAL_OK)
{
    Error_Handler();
}
sConfigIC.ICPolarity = TIM_INPUTCHANNELPOLARITY_RISING;
sConfigIC.ICSelection = TIM_ICSELECTION_DIRECTTI;
sConfigIC.ICPrescaler = TIM_ICPSC_DIV1;
sConfigIC.ICFilter = 0;
if (HAL_TIM_IC_ConfigChannel(&htim3, &sConfigIC, TIM_CHANNEL_2) != HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN TIM3_Init 2 */

/* USER CODE END TIM3_Init 2 */

}

/**
* @brief USART2 Initialization Function
* @param None
* @retval None
*/
static void MX_USART2_UART_Init(void)
{

/* USER CODE BEGIN USART2_Init 0 */

/* USER CODE END USART2_Init 0 */

/* USER CODE BEGIN USART2_Init 1 */

/* USER CODE END USART2_Init 1 */
huart2.Instance = USART2;
huart2.Init.BaudRate = 57600;
huart2.Init.WordLength = UART_WORDLENGTH_9B;

```

```

huart2.Init.StopBits = UART_STOPBITS_1;
huart2.Init.Parity = UART_PARITY_EVEN;
huart2.Init.Mode = UART_MODE_TX_RX;
huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
huart2.Init.OverSampling = UART_OVERSAMPLING_16;
if (HAL_UART_Init(&huart2) != HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN USART2_Init 2 */

/* USER CODE END USART2_Init 2 */

}

/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    /* USER CODE BEGIN MX_GPIO_Init_1 */

    /* USER CODE END MX_GPIO_Init_1 */

    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOC_CLK_ENABLE();
    __HAL_RCC_GPIOH_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOA, LD2_Pin|GPIO_PIN_10, GPIO_PIN_RESET);

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_4, GPIO_PIN_RESET);

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4|GPIO_PIN_5, GPIO_PIN_RESET);

    /*Configure GPIO pin : B1_Pin */
    GPIO_InitStruct.Pin = B1_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(B1_GPIO_Port, &GPIO_InitStruct);

    /*Configure GPIO pins : PA0 PA7 */
    GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_7;
    GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
    GPIO_InitStruct.Pull = GPIO_PULLDOWN;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

    /*Configure GPIO pins : LD2_Pin PA10 */
    GPIO_InitStruct.Pin = LD2_Pin|GPIO_PIN_10;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

    /*Configure GPIO pin : PC4 */
    GPIO_InitStruct.Pin = GPIO_PIN_4;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;

```

```

GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

/*Configure GPIO pins : PB4 PB5 */
GPIO_InitStruct.Pin = GPIO_PIN_4|GPIO_PIN_5;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

/*Configure GPIO pins : PB6 PB8 PB9 */
GPIO_InitStruct.Pin = GPIO_PIN_6|GPIO_PIN_8|GPIO_PIN_9;
GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
GPIO_InitStruct.Pull = GPIO_PULLDOWN;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

/* USER CODE BEGIN MX_GPIO_Init_2 */

/* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */
float CalibrateTemperature(float new_temp)
{
    temp_buffer[temp_index++] = new_temp;

    if (temp_index >= TEMP_BUFFER_SIZE)
    {
        temp_index = 0;
        buffer_full = 1;
    }

    if (!buffer_full) return new_temp;

    // Copy buffer
    float temp_copy[5];
    memcpy(temp_copy, temp_buffer, sizeof(temp_buffer));

    // Sort (simple bubble sort)
    for (int i = 0; i < 4; i++)
        for (int j = i + 1; j < 5; j++)
            if (temp_copy[i] > temp_copy[j])
            {
                float t = temp_copy[i];
                temp_copy[i] = temp_copy[j];
                temp_copy[j] = t;
            }

    // Average middle 3 values
    return (temp_copy[1] + temp_copy[2] + temp_copy[3]) / 3.0f;
}

void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    if ((htim->Instance == TIM3) && (htim->Channel == HAL_TIM_ACTIVE_CHANNEL_2))
    {
        uint32_t capture = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_2);

        uint32_t diff;
        if (capture >= last_capture_time)
            diff = capture - last_capture_time;
        else
            diff = (0xFFFF - last_capture_time + capture + 1);
    }
}

```

```

    last_capture_time = capture;

    if (diff != 0)
    {
        float timer_clock = HAL_RCC_GetPCLK1Freq() * 2.0f;
        float frequency = timer_clock / diff;

        // ✓ Correct conversion
        float temperature = (frequency / 4096.0f) - 50.0f;

        // ✓ Filter AFTER conversion
        current_temp = CalibrateTemperature(temperature);

        temp_ready = 1;
    }
}

/* USER CODE END 4 */

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1)
    {
    }
    /* USER CODE END Error_Handler_Debug */
}

#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number
 * where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and line number,
    ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
    /* USER CODE END 6 */
}

#endif /* USE_FULL_ASSERT */

```

Demo Test 1 Corrections

Monday, 16 March 2026 20:25



Demo 1
Test Corr...

Question 1

Correct

Mark 2.00 out of 2.00

Flag question

[2 marks / 2 punte]

A UART port is configured to operate at 57600 baud, with 8-bit data, one start bit and one stop bit, and even parity bit. What is the maximum data transfer rate on the channel, in bytes per second (rounded to a full byte)?

'n UART poort word opgestel vir dataoordrag teen 57600 baud, met 8 databits, een begin- en een stop-bis en ewe pariteitsbis. Wat is die maksimum dataoordragtempo op die kanaal, in heel grepe per sekonde (gerond tot 'n vol greep)?

Answer: 5236



char frame = 8+1+1+1 = 11 characters

Baud = 57 600

↳ A measure of how fast a communication channel transmits symbols

↳ Bit rate (bps) = Baud rate (baud) × bits per second

Transfer rate per byte = $\frac{57600}{11} = 5236 \text{ byte/s}$

Question 2

Incorrect

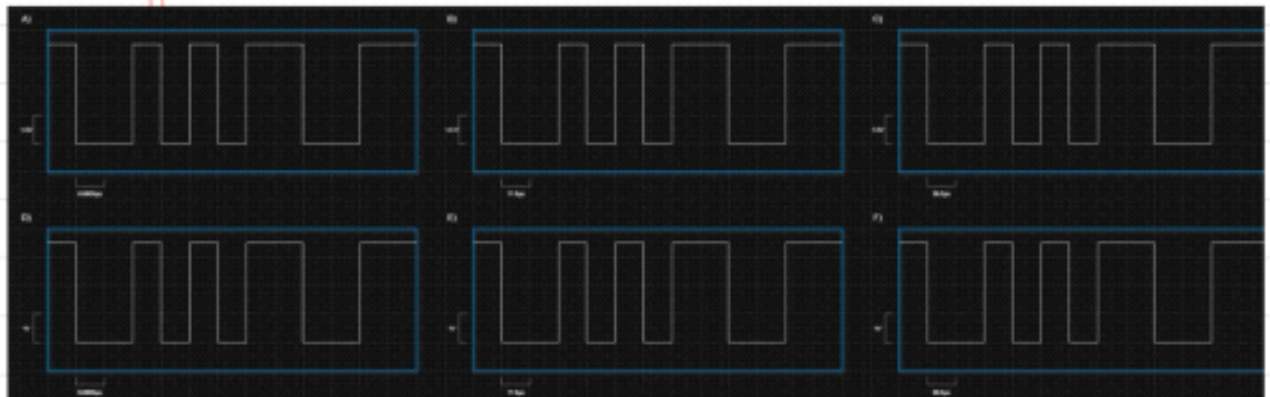
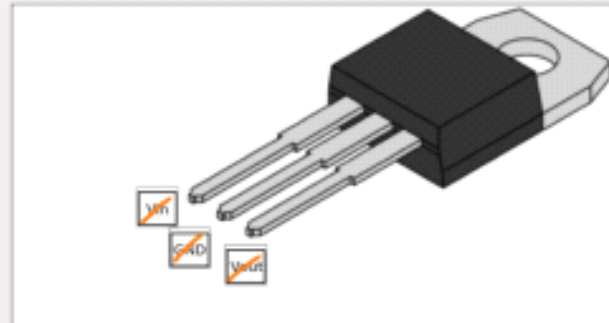
Mark 0.00 out of 1.00

Flag question

[1 mark(s) / 1 punte]

Using the LD1117 datasheet place the pin names in the correct places on the image.

Gebruik die LD1117-datablad en plaas die pinname op die regte plekke op die prent



Question 3

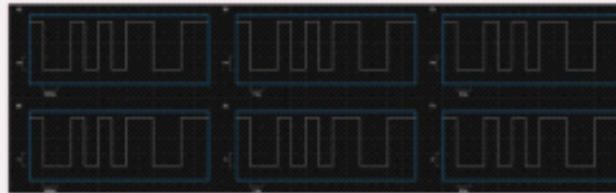
Incorrect

Mark 0.00 out of 3.00

Flag question

[3 marks / 3 puntel

Consider the following set of oscilloscope readings from measuring sent UART data. (Open the image in a new tab to zoom in) // *Onweeg die volgende stel ossilloskooplesings vanaf die meting van gestuurde UART-data: (Maak die prent in 'n nuwe oortjie oop om in te zoom)*



A) Which of these measurements above would you expect to be sent from your Design (E) - 314 project device, if you instantiated the UART settings properly? // *Watter van hierdie metings sou jy verwag om vanaf jou Ontwerp (E) - 314-projektoestel gestuur te word as jy die UART-instellings behoorlik geïnstalleer het?*

☐ 0

$$\text{Baud} = 57600 \Rightarrow \text{Bit time} = \frac{1}{57600} = 17.4 \mu\text{s}$$

11 bits total $\Rightarrow$ 1 start + 8 data + 1 parity (even) + 1 stop
 Always low (0) Even $\rightarrow$ 0 Always high (1)

Idle high $\Rightarrow$ The signal starts high before the start bit

$\therefore$ E

B) Consider now the following UART instantiation variables from the STM32Cube code of another project. // *Onweeg nou die volgende UART-instansiasieveranderlikes van die STM32Cube-kode van 'n ander projek.*

```
huart2.Instance = USART2;
huart2.Init.BaudRate = 115200;
huart2.Init.WordLength = UART_WORDLENGTH_9B;
huart2.Init.StopBits = UART_STOPBITS_1;
huart2.Init.Parity = UART_PARITY_EVEN;
huart2.Init.Mode = UART_MODE_TX_RX;
huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
huart2.Init.OverSampling = UART_OVERSAMPLING_16;
huart2.Init.OneBitSampling = UART_ONE_BIT_SAMPLE_DISABLE;
huart2.Init.ClockPrescaler = UART_PRESCALER_DIV1;
huart2.AdvancedInit.AdvFeatureInit = UART_ADVFEATURE_NO_INIT;
```

Which measurement would you expect to be sent from this project device? // *Watter meting sou jy verwag om vanaf hierdie projektoestel gestuur te word?*

☐ E

$$\text{Baud} = 115200 \Rightarrow \text{Bit time} = \frac{1}{115200} = 8.68 \mu\text{s}$$

$\therefore$ 0

Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char
0	000	000	(null)	32	20	040	SP	64	40	100	@	96	60	140	o
1	001	001	(space)	33	21	041	!	65	41	101	A	97	61	141	p
2	002	002	"	34	22	042	"	66	42	102	B	98	62	142	q
3	003	003	#	35	23	043	#	67	43	103	C	99	63	143	r
4	004	004	\$	36	24	044	\$	68	44	104	D	100	64	144	s
5	005	005	%	37	25	045	%	69	45	105	E	101	65	145	t
6	006	006	&	38	26	046	&	70	46	106	F	102	66	146	u
7	007	007	'	39	27	047	'	71	47	107	G	103	67	147	v
8	008	008	(backspace)	40	28	050	(	72	48	110	H	104	68	150	w
9	009	009	(tab)	41	29	051	)	73	49	111	I	105	69	151	x
10	010	010	\	42	2A	052	[	74	4A	112	J	106	70	152	y
11	011	011	^	43	2B	053	\	75	4B	113	K	107	71	153	z
12	012	012	_	44	2C	054	]	76	4C	114	L	108	72	154	{
13	013	013	`	45	2D	055	^	77	4D	115	M	109	73	155	
14	014	014	~	46	2E	056	_	78	4E	116	N	110	74	156	~
15	015	015	(bELL)	47	2F	057	`	79	4F	117	O	111	75	157	
16	016	016	(LF)	48	30	060	(	80	50	120	P	112	76	160	(
17	017	017	(CR)	49	31	061	)	81	51	121	Q	113	77	161	)
18	018	018	(BS)	50	32	062	[	82	52	122	R	114	78	162	[
19	019	019	(HT)	51	33	063	\	83	53	123	S	115	79	163	]
20	01A	01A	(VT)	52	34	064	^	84	54	124	T	116	80	164	^
21	01B	01B	(FF)	53	35	065	_	85	55	125	U	117	81	165	_
22	01C	01C	(DEL)	54	36	066	(	86	56	126	V	118	82	166	(
23	01D	01D	(SH)	55	37	067	)	87	57	127	W	119	83	167	)
24	01E	01E	(PL)	56	38	070	(	88	58	130	X	120	84	170	(
25	01F	01F	(PR)	57	39	071	)	89	59	131	Y	121	85	171	)
26	020	020	(SO)	58	3A	072	[	90	5A	132	Z	122	86	172	[
27	021	021	(SI)	59	3B	073	\	91	5B	133	[	123	87	173	]
28	022	022	(DLE)	60	3C	074	^	92	5C	134	^	124	88	174	^
29	023	023	(DC4)	61	3D	075	_	93	5D	135	_	125	89	175	_
30	024	024	(DC3)	62	3E	076	(	94	5E	136	(	126	90	176	(
31	025	025	(DC2)	63	3F	077	)	95	5F	137	)	127	91	177	)

Number systems

→ Decimal numbers

→ Binary numbers

→ Hexadecimal numbers

→ Octal numbers

What is the ASCII character being transmitted via UART in all the readings?

// Wat is die ASCII-karakter wat in al die lesings via UART oorgelees word?

AB

In image: 0 01010 110 01
 Measurement: 0110 1010
 ∴ 6A

Question 4

Partially correct

Mark 1.00 out of 2.00

Flag question

[2 Marks // 2 Punte]

Match the given Git command to its corresponding description from the provided options. // Pas die gegewe Git-opdrag by die ooreenstemmende beskrywing uit die verskafte opsies.

git add [file]	add a file as it looks now to your next commit (stage) // voeg 'n lêer soos dit is by jou volgende commit
git status	show the commit history for the currently active branch // wys die toekenningsgeskiedenis
git clone [url]	retrieve an entire repository from a hosted location via URL // haal 'n volledige repositorie van 'n gehoste plek via URL
git branch	show the commit history for the currently active branch // wys die toekenningsgeskiedenis

1. git add [file]

→ add a file as it looks now to your next commit (stage)

2. git status

→ show the status of the working tree and staging area

3. git clone [url]

→ retrieve an entire repository from a hosted location via URL

4. git branch

→ show the commit history for the current

Question 5

Incorrect
Mark 0.00 out of 2.00

Flag question

12 Marks / 2 Punte

For your D2 LED used in the project, what is the primary design current limit to still have 3.3V across the LED and resistor combination?

Vir jou D2 LED wat in die projek gebruik word, wat is die primêre ontwerpstroomlimiet om steeds 3.3V oor die LED en resistor kombinasie te hê?

Select one:

- ☐ a. GPIO absolute maximum current as stated in the STM32 datasheet / GPIO absolute maksimum stroom soos vermeld in die STM32 datablad
- ☐ b. The usable brightness level of the LED as observed by the designer / Die bruikbare helderheidsvlak van die LED soos waargeneem deur die ontwerper
- ☒ c. LED operational current as stated in its datasheet / LED operasionele stroom soos vermeld in sy datablad
- ☐ d. LED absolute maximum current as stated in its datasheet / LED absolute maksimum stroom soos aangedui in sy datablad
- ☐ e. GPIO operational current as stated in the STM32 datasheet / GPIO operasionele stroom soos vermeld in die STM32 datablad

In the project, the LED is driven from an STM32 GPIO pin.

↳ The GPIO is the limiting component, not the LED

So the maximum safe current is determined by what the STM32 GPIO can source or sink continuously, not by what the LED could theoretically handle

Question 6

Partially correct
Mark 1.00 out of 2.00

Flag question

12 marks / 2 punte

You tested your project on your bench using 5V from the bench supply and the USB cable connected to a terminal program set to 8E1 and 57600 BAUD. You got the correct system responses during these tests. When you demo your board, there is no communication between your device and the teststation. Mark all of the following options that could cause the problem.

Jy het jou projek op jou bank getoets met 5V vanaf die banktoevoer en die USB-kabel gekoppel aan 'n terminaalprogram wat op 8E1 en 57600 BAUD gestel is. Jy het die korrekte antwoorde van jou stelsel tydens hierdie toetse gekry. Wanneer jy jou bord demonstree, is daar geen kommunikasie tussen jou toestel en die toetsstasie nie. Merk al die volgende opsies wat die probleem kan veroorsaak.

Select one or more:

- ☐ a. Your STM32 Jumper is set to 5V and not 5.5V / Jou STM32-jumper is op 5V gestel en nie 5.5V nie
- ☒ b. The UART parity bit is set incorrectly / Die UART-pariteitbit is verkeerd gestel
- ☐ c. The 3.3V regulator is soldered in the wrong way round / Die 3.3V-reguleerder is verkeerd gesoldeer
- ☐ d. The HAL_UART_Transmit() function is never called / Die HAL_UART_Transmit() funksie word nooit genoem nie
- ☒ e. The RX and TX lines are not making a good connection / Die RX- en TX-lyne maak nie 'n goeie verbinding nie
- ☒ f. The RX and TX lines are not wired to the TIC / Die RX- en TX-lyne is nie aan die TIC bedraad nie
- ☒ g. The UART TX and RX lines are swapped / Die UART TX- en RX-lyne word omgeruil
- ☐ h. The UART BAUD rate is set incorrectly / Die UART BAUD-koers is verkeerd gestel
- ☐ i. The jumper J30 is not soldered in / Die konneksie J30 is nie ingesoldeer nie
- ☐ j. The HAL_UART_Receive() function is never called / Die HAL_UART_Receive() funksie word nooit genoem nie

Question 7

Incorrect

Mark 0.00 out of 2.00

 Flag question

An STM32 MCU runs at 84 MHz.

An LMT01 sensor outputs pulses at 88 kHz.

The LMT01 is connected to a GPIO pin as an external interrupt on the STM32 (rising edge triggered).

How many full MCU clock cycles occur during one LMT01 sensor pulse period?

//

In STM32 MCU loop teen 84 MHz. In LMT01-sensor gee pulse teen 88 kHz uit. Die LMT01 is aan 'n GPIO-pen gekoppel as 'n ekstern onderbreking op die STM32 (stigende rand geaktiveer). Hoeveel volle MCU-kloksiklusse vind plaas gedurende een LMT01-sensorpulsperiode?

Answer: 954.5



cycles per pulse =

$$\text{clock cycles} = \frac{84 \times 10^6}{88 \times 10^3} = 954$$

When LMT01 pulses once, STM32 will have cycled through 954 times



Computer
Systems 2...

Computer Systems 214

Number systems

Lecture 2

Dr WD Duckitt & Dr HW Jordaan

COPYRIGHT

Copyright © 2023 Stellenbosch University
All rights reserved

DISCLAIMER

This content is provided without warranty or representation of any kind. The use of the content is entirely at your own risk and Stellenbosch University (SU) will have no liability directly or indirectly as a result of this content.

The content must not be assumed to provide complete coverage of the particular study material. Content may be removed or changed without notice.

The video is of a recording with very limited post-recording editing. The video is intended for use only by SU students enrolled in the particular module.

Lecture Overview

- What is a number system?
- Decimal numbers *0 - 9 digits*
- Binary numbers
- Hexadecimal numbers
- Octal numbers
- Conversion between number systems
- Bytes, Nibbles, LSB, MSB, etc.
- Representing letters (ASCII table)

Relevant section in textbook: 1.4.1-1.4.4

Computer Systems 214

3

Number systems intro

- Refers to the numerical representation of a value.
- A good number system will:
 - Represent a useful set of numbers (e.g. integers, or rational numbers)
 - Give every number represented a unique representation
 - Reflect the algebraic and arithmetic structure of the numbers
- In digital logic we only work with two states – HIGH and LOW.
- It is clumsy to work with numbers between 0 – 9 (decimal) to perform mathematics with a two state (binary) system.
- We use different number systems to make life easier when working with and designing digital systems.

Computer Systems 214

4

Decimal numbers

- Just as you (probably) have ten fingers, there are ten decimal digits from 0 to 9. [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
- Each digit (right to left) represents a multiple of a power of 10.
- Decimal numbers are also called **base-10** (or radix-10) numbers.

max 9

1000's	100's	10's	1's
9	7	4	2

Indicating the **base** of the number

$$9742_{10} = 9 \times 10^3 + 7 \times 10^2 + 4 \times 10^1 + 2 \times 10^0$$

nine thousands
seven hundreds
four tens
two ones

- An N-digit decimal number represents one of 10^N possibilities. This is called the **range** of the number.
 - Three-digit decimal number represents one of 1000 possibilities in the range of 0 to 999.

Computer Systems 214

5

Binary numbers

- Binary digits or "bits" represent one of two values, 0 or 1.
- Each bit (right to left) represent a multiple of a power of 2.
- Binary numbers are **base-2**.

max 15

16's	8's	4's	2's	1's
1	0	1	1	0

$$10110_2 = 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 22_{10}$$

one sixteen
no eight
one four
one two
no one

- An N-bit binary number represents one of 2^N possibilities.
 - A four-bit binary number represents one of 16 possibilities. For positive numbers, this is the range of 0 to 15.
- Binary numbers are convenient for logic circuits in computers, since they can represent 2 states, i.e. low and high voltages.

Computer Systems 214

6

Number interpretation

- The interpretation of a number depends on the base number used:

Decimal: $1101 = 1 \times 10^3 + 1 \times 10^2 + 0 \times 10^1 + 1 \times 10^0$
 $= 1000 + 100 + 1$
 $= \text{one thousand, one hundred and one}$

Binary: $1101 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$
 $= 8 + 4 + 1$
 $= 13$
 $= \text{thirteen}$

- The base number, or radix, is indicated with a subscript when there is potential for confusion:

$$(1101)_2 = (13)_{10}$$

Computer Systems 214

7

Conversion between Decimal and Binary

Convert 84_{10} to binary $\rightarrow$ go to base 2

Solution 1 (Left to Right)

Find largest power of 2 less than or equal to the number.
 Subtract and repeat until 0.

2 ^{biggest number} $1 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$

Answer

$$1010100_2 \quad \begin{array}{l} 84-64 \\ = 20 \end{array}$$

$$20-16$$

$$4$$

$$4-4=0$$

3 $\Rightarrow 1010100$

$$1 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 \Rightarrow \text{sum together to get back to decimal}$$

1 $2^0 = 1$

$2^1 = 2$

$2^2 = 4$

$2^3 = 8$

$2^4 = 16$

$2^5 = 32$

$2^6 = 64$

$2^7 = 128$

Computer Systems 214

8

Conversion between Decimal and Binary

Convert 84_{10} to binary

Solution 2 (Right to Left)

Repeatedly divide the number by 2, the remainder goes in each column.

Answer

1010100_2

Hexadecimal numbers *max 15*

- Decimal numbers are useful, because they are used by people.
- Binary numbers are useful, because they are used by computers.
- Hexadecimal (**base-16**) numbers are useful, because they provide a shorthand notation for binary numbers.
- Each hexadecimal digit represents a group of four bits (a nibble).
- Hexadecimal numbers use the **digits 0 to 9** along with the **letters A to F**, to represent a total of 16 possible values (**from 0 to 15**).
- Each digit (right to left) represents a multiple of a power of 16.

256's	16's	1's
2	E	D

$$\rightarrow 2ED_{16} = 2 \times 16^2 + E \times 16^1 + D \times 16^0 = 749_{10}$$

two two hundred fifty six's fourteen sixteens thirteen ones

Conversion table

Table 1.2 Hexadecimal number system

Hexadecimal Digit	Decimal Equivalent	Binary Equivalent
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Computer Systems 214

11

Binary to hexadecimal conversion

- A binary number is converted to a hexadecimal number by taking groups of four bits, starting from the bit in the 1's column, and replacing them with the corresponding hexadecimal digits.
- A 16-bit binary number can be represented with four hex digits, for example:

$$(1010111100100101)_2 = (AF25)_{16}$$

because 1010 1111 0010 0101



A F 2 5

- If the number of bits is not a multiple of four, then add 0's to the left of the binary number.
- Conversion from hexadecimal to binary is also straightforward: replace each hexadecimal digit by four bits that represent the same value

Computer Systems 214

12

Conversion between Decimal and Hexadecimal

Convert 333_{10} to hexadecimal

Solution 1 (Left to Right)

Find largest power of 16 less than or equal to the number.
Subtract and repeat until 0.

Answer $1 \times 16^2 + 4 \times 16^1 + D \times 16^0$
 $14D_{16}$ $333 - 256 = 77$ $77 - 64 = 13$ $\Rightarrow 14D$

$$\begin{aligned} 16^0 &= 1 \\ 16^1 &= 16 \\ 16^2 &= 256 \\ &\dots \end{aligned}$$

Conversion between Decimal and Hexadecimal

Convert 333_{10} to hexadecimal

Solution 2 (Right to Left)

Repeatedly divide the number by 16, the remainder goes in each column.

Answer

$14D_{16}$

Octal numbers

- Octal (**base-8**) numbers uses the **digits from 0 to 7**. A less used alternative to writing binary in shorthand.
- A 12-bit binary number can be represented with four octal digits, for example:

$$(111100100101)_2 = (7445)_8$$

because 111 100 100 101
7 4 4 5

- Why do engineers confuse Halloween with Christmas?
⇒ 31 OCT = 25 DEC

Computer Systems 214

15

Bytes, Nibbles, LSB, MSB

bit
1 0 1 1 0 0 1

- Each digit in a binary number is called a **bit**.
- A group of *eight bits* is called a **byte**. ($2^8=256$ possible values)
- A group of *four bits* is called a **nibble**. ($2^4=16$ possible values)
- Microprocessors handle data in chunks called **words**. The size of a word depends on the architecture of the microprocessor.
 - E.g. 64-bit processor operates in 64-bit words.
- The bit in the 1's column is called the **least-significant bit (lsb)**
- The bit on the other end is called the **most-significant bit (msb)**
- Similarly, within a word, the bytes are identified as the **least significant byte (LSB)** through **most significant byte (MSB)**



NB

Computer Systems 214

16

Number sizes

- By handy coincidence:

- $2^{10} = 1024 \approx 10^3 \Rightarrow$ **kilo** (upper case K to disambiguate from k =1000)

- $2^{20} = 1048576 \approx 10^6 \Rightarrow$ **mega**

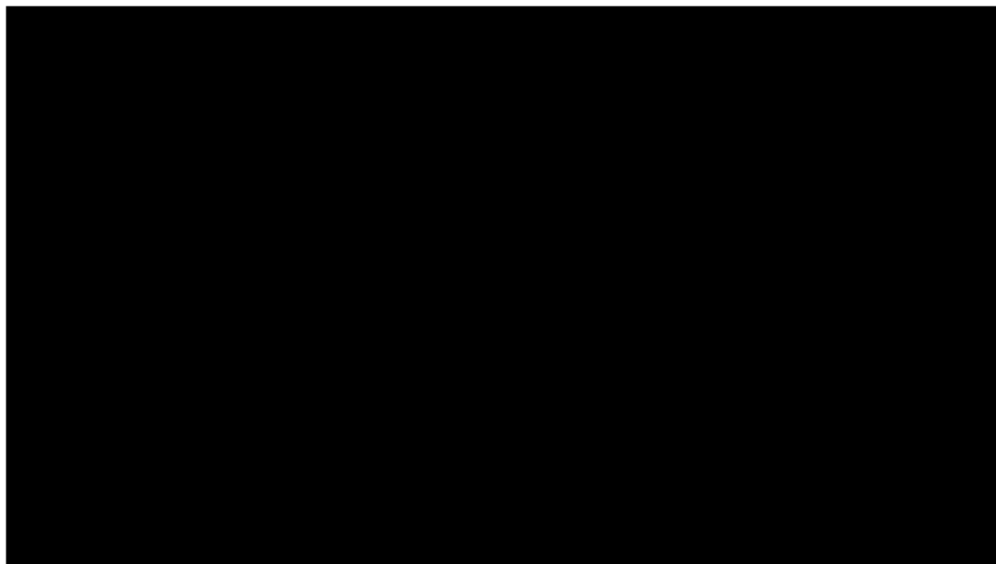
- $2^{30} = 1073741824 \approx 10^9 \Rightarrow$ **giga**

- 1024 bytes is called a **kilobyte (KB)**

- 1024 bits is called a **kilobit (Kb)** (note lowercase b)

- Similarly MB, Mb, GB, Gb...

Representing letters (ASCII)



Source: <https://www.youtube.com/watch?v=1GSjbWt0c9M>

ASCII table

Table 6.5 ASCII encodings

#	Char	#	Char	#	Char	#	Char	#	Char	#	Char
20	space	30	0	40	@	50	P	60	`	70	p
21	!	31	1	41	A	51	Q	61	a	71	q
22	"	32	2	42	B	52	R	62	b	72	r
23	#	33	3	43	C	53	S	63	c	73	s
24	\$	34	4	44	D	54	T	64	d	74	t
25	%	35	5	45	E	55	U	65	e	75	u
26	&	36	6	46	F	56	V	66	f	76	v
27	'	37	7	47	G	57	W	67	g	77	w
28	(	38	8	48	H	58	X	68	h	78	x
29	)	39	9	49	I	59	Y	69	i	79	y
2A	*	3A	:	4A	J	5A	Z	6A	j	7A	z
2B	+	3B	;	4B	K	5B	[	6B	k	7B	{
2C	,	3C	<	4C	L	5C	\	6C	l	7C	
2D	-	3D	=	4D	M	5D	]	6D	m	7D	}
2E	.	3E	>	4E	N	5E	^	6E	n	7E	~
2F	/	3F	?	4F	O	5F	_	6F	o		

Computer Systems 214

19

Declaring/Printing C ints in Different Number Systems

- In C the variable type `int` is stored as binary number of at least 16 bits
- Declaration and presentation may be expressed in higher-base number systems, with varying syntax
- Declaration:
 - `int x = 11;` declares `x` as the number 11 in base-10 (decimal)
 - `int x = 0b1011;` declares `x` as the same value in base-2 (binary)
 - `int x = 013;` declares `x` as the same value in base-8 (octal)
 - `int x = 0xB;` declares `x` as the same value in base-16 (hexadecimal)
- Presentation:
 - `printf("%d", x);` prints `x` in base-10 (decimal)
 - `printf("%o", x);` prints `x` in base-8 (octal)
 - `printf("%x", x);` prints `x` in base-16 (hexadecimal)
- NB: There is no standardised C syntax to print in base-2 (binary)

NB exam Q to code

Computer Systems 214

20

Declaring/Printing C ints in Different Number Systems

```
1 #include <stdio.h> /* printf */
2 #include "dtob.h" /* decimal_to_binary */
3
4 int main()
5 {
6     // Decimal number
7     int x = 17;
8     printf("Decimal: %d\n", x);
9     printf("Binary: %s\n", decimal_to_binary(x));
10    printf("Hexadecimal: %x\n\n", x);
11
12    // Binary number
13    int y = 0b1011;
14    printf("Decimal: %d\n", y);
15    printf("Binary: %s\n", decimal_to_binary(y));
16    printf("Hexadecimal: %x\n\n", y);
17
18    // Hexadecimal number
19    int z = 0xbf;
20    printf("Decimal: %d\n", z);
21    printf("Binary: %s\n", decimal_to_binary(z));
22    printf("Hexadecimal: %x\n\n", z);
23
24    return 0;
25 }
```

Decimal:	17
Binary:	00010001
Hexadecimal:	11
Decimal:	11
Binary:	00001011
Hexadecimal:	b
Decimal:	191
Binary:	10111111
Hexadecimal:	bf

Computer Systems 214

21

Next lecture...

- Signed numbers (positive and negative numbers)
- Addition and subtraction of binary numbers

Computer Systems 214

22

Concluding remark

There are 10 kinds of people in the world: Those who understand binary and those who don't.

Demo 2 Test Notes

Monday, 16 March 2026 20:29

Components that were Implemented during Demo 2

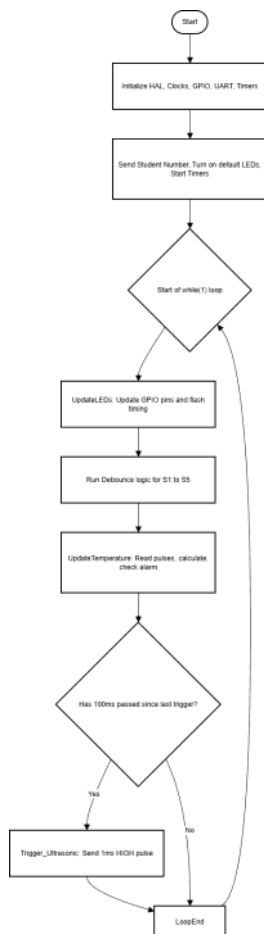
- Ultrasonic sensor

Software Implementation during Demo 2

- Ultrasonic sensor
- Stat command

Main Super Loop Flowchart

Monday, 16 March 2026 21:36



1. Initialization (Before the Loop)

Before the loop even begins, the microcontroller must set up its hardware peripherals.

- **Boot & Configuration:** The system configures the core clock, initializes the Hardware Abstraction Layer (HAL), and sets up the GPIO pins, UART communication, and Timers.
- **Startup Sequence:** The system sends the student number over UART, turns on the default LEDs, and starts the timer interrupts so they are ready to catch incoming signals.

2. Entering the while(1) Loop

Once setup is complete, the program enters the infinite loop. The code inside this loop executes as fast as the CPU can run it, repeating thousands of times per second.

Importantly, your loop is **non-blocking**. This means it never uses `HAL_Delay()` inside the loop, which would freeze the CPU. Instead, it uses `HAL_GetTick()` to check timestamps and run tasks only when it is their turn.

3. Step-by-Step Execution Inside the Loop

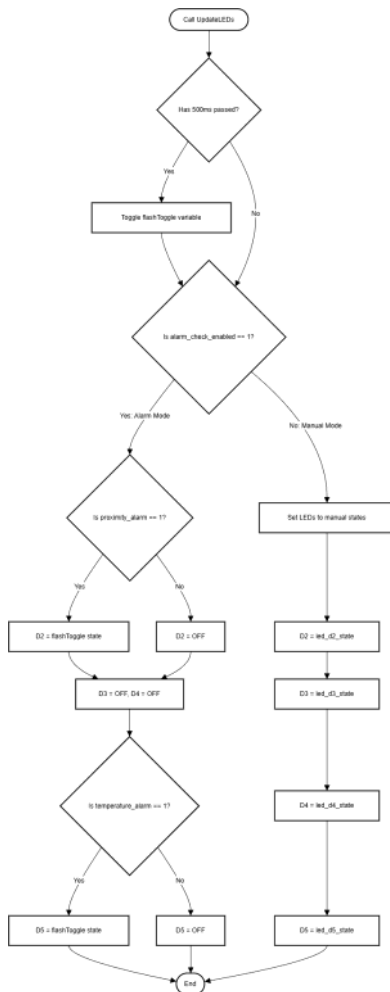
- **Task A: UpdateLEDs()**
 - **What it does:** It checks if 500 milliseconds have passed to toggle a `flashToggle` variable.
 - **Why:** If the system is in an "alarm state", it uses this toggle to flash specific LEDs (D2 for proximity, D5 for temperature). If not in an alarm state, it simply applies the manual LED states set by the user buttons.
- **Task B: Button Polling (S1_Button to S5_Button)**
 - **What it does:** The loop rapidly checks the physical voltage level (HIGH/LOW) on the GPIO pins connected to your 5 buttons.
 - **Why:** It uses a state machine and timestamps to "debounce" the buttons. It verifies that a button was held down for at least 50ms before registering a valid press. This prevents one physical press from registering as multiple rapid presses due to mechanical noise.
- **Task C: UpdateTemperature()**
 - **What it does:** It checks if exactly 1000 milliseconds (1 second) have passed since it last updated the temperature.
 - **Why:** If a second has passed, it takes the number of pulses counted by the Timer 3 interrupt in the background, calculates the actual temperature, runs it through your calibration function, and checks if the temperature exceeds 30.0°C to trigger an alarm. If a second has *not* passed, it skips this step instantly.
- **Task D: Ultrasonic Trigger Check**
 - **What it does:** It checks if 100 milliseconds have passed since the last time it pinged the ultrasonic sensor.
 - **Why:** The ultrasonic sensor needs time for the sound wave to travel and return. If 100ms have passed, it calls `Trigger_Ultrasonic()` to send a 1-millisecond HIGH pulse to the sensor's trigger pin, commanding it to take a new measurement. (The actual reading of the distance happens in the background via the Timer 2 interrupt).

4. Loop Back to Start

Once the code reaches the bottom of the while(1) block, it immediately jumps back to the top and starts over. Because tasks C and D are time-gated, the loop spends 99% of its time just rapidly updating LEDs and checking for user button presses, resulting in a highly responsive system.

UpdateLEDs Function Flowchart

Monday, 16 March 2026 22:15



Expanded Explanation: LED Operations (UpdateLEDs)

This function acts as the "display driver" for your microcontroller. It runs constantly in the super loop and dictates exactly what the physical LEDs should be doing at any given millisecond.

• Step 1: The Heartbeat Timer (Flash Toggle)

- **What it does:** The code checks if 500 milliseconds have passed since the last time it flipped the flashToggle variable. If yes, it toggles it (from 1 to 0, or 0 to 1).
- **Why:** This creates a universal, continuous 1Hz blinking reference (500ms ON, 500ms OFF) running in the background. Because it uses HAL_GetTick(), it doesn't block the CPU. Any LED that needs to blink can simply copy the current state of flashToggle.

• Step 2: The Master Mode Check

- **What it does:** The code evaluates the global alarm_check_enabled flag to decide which branch of logic to follow.
- **Why:** The LEDs have dual purposes. The system needs to know whether it should display **sensor warnings** (Armed Mode) or **user-defined static states** (Manual Mode).

• Step 3: Executing "Alarm Mode"

- **What it does:** If the system is armed (alarm_check_enabled == 1), the code takes full control of the LEDs, overriding the user's manual settings.
 - It forces D3 and D4 completely OFF.
 - It checks proximity_alarm. If active, it links D2 to the flashToggle heartbeat (making it blink). If safe, D2 is forced OFF.
 - It checks temperature_alarm. If active, it links D5 to the flashToggle heartbeat. If safe, D5 is forced OFF.
- **Why:** When monitoring for danger, you want the display to be unambiguous. Forcing unrelated LEDs off ensures the user clearly sees the flashing warning lights if a sensor threshold is breached.

• Step 4: Executing "Manual Mode"

- **What it does:** If the system is NOT armed, it ignores all alarm flags. It simply maps the physical GPIO pins directly to their corresponding software variables (led_d2_state, led_d3_state, etc.).
- **Why:** This returns control to the user. If the user pressed S1 to turn D2 ON, the code sees led_d2_state == 1 and writes the pin HIGH continuously (no flashing).

Ultrasonic sensor code

Monday, 16 March 2026 20:40

```
//Variables for the proximity sensor
volatile uint32_t echo_start = 0;
volatile uint32_t echo_end = 0;
volatile uint8_t echo_captured = 0;
volatile float distance_cm = 0.0f;
```

1. The Trigger Phase

To start a measurement, an ultrasonic sensor requires a short digital HIGH pulse on its Trigger pin (typically at least 10 microseconds). This tells the sensor to emit an ultrasonic sound burst (usually 8 cycles at 40kHz).

```
void Trigger_Ultrasonic(void)
{
    HAL_GPIO_WritePin(TRIG_PORT, TRIG_PIN, GPIO_PIN_SET);
    HAL_Delay(1); // guarantees >10us pulse
    HAL_GPIO_WritePin(TRIG_PORT, TRIG_PIN, GPIO_PIN_RESET);
}
```

- The code sets PC3 (TRIG_PIN) HIGH.
- It uses HAL_Delay(1) to wait for 1 millisecond. While 1ms is much longer than the required 10µs, it safely guarantees the sensor registers the trigger.
- The pin is then pulled LOW.
- In the main while(1) loop, this function is called every 100 milliseconds to continuously sample the distance.

2. The Echo Phase (Input Capture)

After the sound wave bounces off an object and returns, the sensor sets its Echo pin HIGH for a duration exactly equal to the time it took for the sound wave to travel out and back. To measure this precisely without tying up the CPU, your code uses **Timer Input Capture**.

In your code: You have configured Timer 2 (htim2) Channel 2 to trigger an interrupt whenever the state of the Echo pin changes. This is handled inside the HAL_TIM_IC_CaptureCallback function.

```
// PROXIMITY SENSOR (HC-SR04)
if ((htim->Instance == TIM2) && (htim->Channel == HAL_TIM_ACTIVE_CHANNEL_2))
{
    uint32_t capture = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_2);

    if(echo_captured == 0)
    {
        echo_start = capture;
        echo_captured = 1;
        __HAL_TIM_SET_CAPTUREPOLARITY(htim, TIM_CHANNEL_2, TIM_INPUTCHANNELPOLARITY_FALLING);
    }
    else
    {
        echo_end = capture;
        echo_captured = 0;
        __HAL_TIM_SET_CAPTUREPOLARITY(htim, TIM_CHANNEL_2, TIM_INPUTCHANNELPOLARITY_RISING);
        uint32_t diff;
        if(echo_end >= echo_start)
            diff = echo_end - echo_start;
        else
            diff = (0xFFFFFFFF - echo_start + echo_end + 1);

        float time_us = diff;
        distance_cm = time_us * 0.0343f / 2.0f;
        proximity_alarm = (alarm_check_enabled && distance_cm < 10.0f);
    }
}
```

- **Rising Edge:** When the Echo pin goes HIGH, the timer captures the current time (echo_start). The code then dynamically reconfigures the timer to look for a falling edge.
- **Falling Edge:** When the Echo pin goes LOW, the timer captures the time again (echo_end).

3. The Math (Distance Calculation)

Once the start and end times are captured, the code calculates the duration of the pulse. Since the timer ticks in microseconds (based on your system clock and prescaler configuration of 83), the difference between echo_end and echo_start is the time in

microseconds(time_us).

```
if(echo_end >= echo_start)
    diff = echo_end - echo_start;
else
    diff = (0xFFFFFFFF - echo_start + echo_end + 1);

float time_us = diff;
distance_cm = time_us * 0.0343f / 2.0f;
proximity_alarm = (alarm_check_enabled && distance_cm < 10.0f);
```

- **Timer Overflow Protection:** The if/else statement ensures that if the 32-bit timer resets to 0 in the middle of a pulse, the math doesn't break.
- **Speed of Sound:** The speed of sound in air is approximately 343 meters per second, which converts to **0.0343 centimeters per microsecond**.
- **Divide by 2:** Because the sound wave travels *to* the object and *back*, the total time represents twice the actual distance. Dividing by 2 gives the straight-line distance to the object.

4. Integration into the System

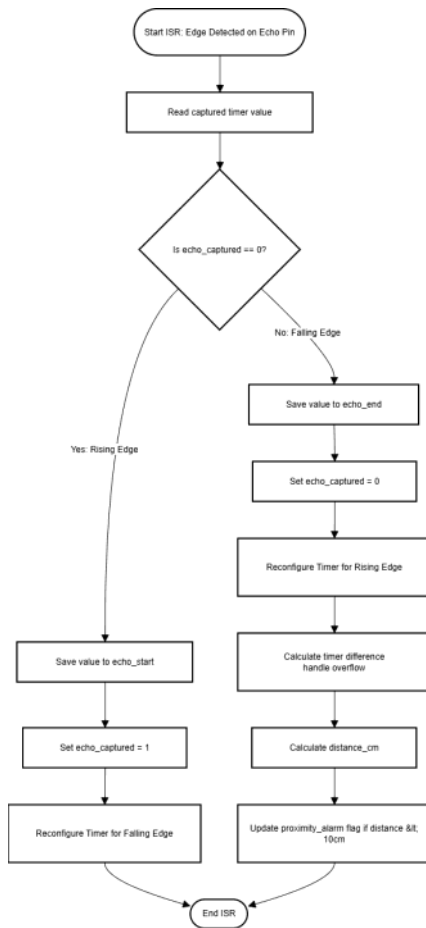
Finally, the newly calculated distance_cm is evaluated to see if it breaches a safety threshold.

```
proximity_alarm = (alarm_check_enabled && distance_cm < 10.0f);
```

If the system is armed (alarm_check_enabled == 1) and an object is closer than 10 cm, the proximity_alarm flag is raised, which subsequently triggers an LED (D2) to flash inside the UpdateLEDs() function and updates the UART @Stat command output.

Ultrasonic Sensor Flowchart

Monday, 16 March 2026 21:45



The Problem with Software Timing

When an ultrasonic sensor (like the HC-SR04) receives a trigger pulse, it emits a sound wave and instantly sets its ECHO pin **HIGH**. Once the sound wave bounces off an object and returns, the sensor sets the ECHO pin **LOW**.

The length of time the pin stays HIGH is exactly proportional to the distance of the object. If the CPU tries to measure this time using a while loop (polling), it is highly inaccurate and wastes processing power.

The Solution: Hardware Input Capture

Instead of the CPU counting the time, your code assigns a **Hardware Timer (TIM2)** to monitor the ECHO pin. "Input Capture" means the timer runs continuously in the background. The exact microsecond a specific physical electrical edge (rising or falling) occurs on the pin, the hardware instantly "captures" (saves) the current timer value into a register and fires an interrupt.

Here is the step-by-step logic inside your HAL_TIM_IC_CaptureCallback function:

Step 1: Catching the Rising Edge (Start of the Echo)

Before a measurement happens, Timer 2 is configured to look for a **Rising Edge** (the transition from LOW to HIGH).

1. **The Edge Occurs:** The sensor sets the ECHO pin HIGH. The timer instantly captures the time (e.g., Timer = 10,500).
2. **The ISR Runs:** The CPU jumps to the interrupt service routine.
3. **State Check:** The code checks if(echo_captured == 0). Since it is 0, the system knows this is the **start** of the pulse.
4. **Save the Time:** It saves 10,500 into the echo_start variable.
5. **Flip the Flag & Reconfigure:** It sets echo_captured = 1 and, crucially, reconfigures the timer's hardware polarity to now look for a **Falling Edge** (TIM_INPUTCHANNELPOLARITY_FALLING).
6. **Exit:** The interrupt ends, and the CPU goes back to the main loop while the sound wave travels through the air.

Step 2: Catching the Falling Edge (End of the Echo)

1. **The Edge Occurs:** The echo returns, and the sensor pulls the ECHO pin LOW. The timer captures the new time (e.g., Timer = 11,500).
2. **The ISR Runs:** The CPU jumps back to the interrupt routine.
3. **State Check:** It checks if(echo_captured == 0). Because it was set to 1 in the previous step, the code jumps to the else block, knowing this is the **end** of the pulse.
4. **Save the Time:** It saves 11,500 into the echo_end variable.
5. **Reset & Reconfigure:** It sets echo_captured = 0 and sets the timer back to look for a **Rising Edge** for the next measurement.

Step 3: The Math and Timer Overflow Protection

Now the system has the start and end times, but it needs to calculate the duration.

- **Normal Calculation:** If echo_end is greater than echo_start (e.g., 11,500 - 10,500), the duration is simply 1,000 microseconds.
- **Timer Overflow Calculation:** Hardware timers eventually hit their maximum value and roll back to 0. If echo_start was 4,294,967,290 (near the 32-bit limit) and echo_end is 100 (it rolled over), a simple subtraction would result in a massive negative/incorrect number. Your code handles this elegantly: $\text{diff} = (0xFFFFFFFF - \text{echo_start} + \text{echo_end} + 1)$.

Step 4: Converting Time to Distance

Finally, the code converts the raw microsecond difference into centimeters:

C

```
distance_cm = time_us * 0.0343f / 2.0f;
```

- **0.0343f:** The speed of sound in air is roughly 343 meters per second, which is 0.0343 centimeters per microsecond. Multiplying the time by this constant gives the total distance the sound traveled.
- **Divide by 2:** Because the sound had to travel *to* the object and *back* to the sensor, the calculated distance is twice the actual distance. Dividing by 2 gives the true distance to the object.

Step 5: Alarm Evaluation

In the very last step of the interrupt, the code checks if this new distance_cm warrants a safety warning. If the system is armed (alarm_check_enabled == 1) and the object is closer than 10.0 cm, it sets proximity_alarm =

1. The main loop will notice this flag on its next cycle and flash the D2 LED.

Stat Command

Monday, 16 March 2026 20:42

```
// For the UART @Stat Command
uint8_t rx;
char uart_cmd[32];
uint8_t idx = 0;
```

1. Interrupt-Driven Reception

In embedded systems, constantly checking if a new character has arrived over UART (polling) wastes CPU cycles and can cause you to miss incoming data if the CPU is busy elsewhere (like calculating temperatures or ultrasonic distances).

Instead, your code uses interrupts. In the main() function, you prime the UART to listen for exactly 1 byte of data in the background:

```
// The stat command
HAL_UART_Receive_IT(&huart2,&rx,1);
```

When a character is received from the PC/terminal, the hardware triggers an interrupt, pausing the main loop and automatically jumping to the HAL_UART_RxCpltCallback function.

2. Buffering and Delimiter Detection

Because the user types commands one character at a time (e.g., 'S', 't', 'a', 't', '\n'), the microcontroller needs to collect these characters into a string before it can understand them.

In your code:

```
if(idx < sizeof(uart_cmd)-1)
{
    uart_cmd[idx++] = rx;
}
```

- **Buffering:** The code places each incoming character (rx) into a character array (uart_cmd).
- **Buffer Overflow Protection:** The if(idx < sizeof(uart_cmd)-1) condition ensures that if a user types a command that is too long, it won't write past the end of the array, which would cause memory corruption (a buffer overflow).
- **Delimiter Check:** The code checks if the incoming character is a newline character (\n). The newline acts as the "End of Message" delimiter, signaling that the user has pressed "Enter" and the command is complete.

3. Command Parsing

Once the newline character is detected, the buffer is treated as a complete string.

In your code:

```
if(rx == '\n')
{
    uart_cmd[idx] = '\0';
    if(strstr(uart_cmd,"Stat"))
    {
        char msg[350];
        snprintf(msg, sizeof(msg),
            "@2026/02/26 22:12:42\n"
            "Distance: %.1f cm\n"
            "Temperature: %.1f C\n"
            "Light: 0000 lux\n"
            "X accel: 0.00 g\n"
            "Y accel: 0.00 g\n"
            "Z accel: 0.00 g\n"
            "Unsafe driving: 0\n"
            "Impact detected: 0\n"
            "Low-Light warning: 0\n"
            "Proximity warning: %d\n"
            "High Temperature: %d\n"
            "GPS lat: 0.00000\n"
            "GPS long: 0.00000\n"
            "&\n",
            distance_cm, current_temp, proximity_alarm, temperature_alarm);

        HAL_UART_Transmit(&huart2,(uint8_t*)msg,strlen(msg),100);

        idx = 0;
    }
}
```

- **Null-Termination:** In C, strings must end with a null character (\0). By appending this to the end of the received characters, standard C string libraries can now safely read it.
- **strstr function:** The code uses strstr (String Substring) to search inside the uart_cmd buffer for the sequence "Stat". If it finds it, it proceeds to generate the response.

4. Response Generation and Formatting

Once the system knows the user wants the "Stat" (Status) report, it needs to gather the real-time sensor variables and inject them into a formatted text block.

In your code:


```

char msg[350];
snprintf(msg, sizeof(msg),
"@2026/02/26 22:12:42\n"
"Distance: %.1f cm\n"
"Temperature: %.1f C\n"
"Light: 0000 lux\n"
"X accel: 0.00 g\n"
"Y accel: 0.00 g\n"
"Z accel: 0.00 g\n"
"Unsafe driving: 0\n"
"Impact detected: 0\n"
"Low-Light warning: 0\n"
"Proximity warning: %d\n"
"High Temperature: %d\n"
"GPS lat: 0.00000\n"
"GPS long: 0.00000\n"
"&\n",
distance_cm, current_temp, proximity_alarm, temperature_alarm);

```

- **snprintf:** This function is essentially printf but instead of printing to a screen, it "prints" into the msg character array.
- It takes the format specifiers (like %.1f for floats with 1 decimal place, and %d for integers) and replaces them with the live volatile variables (distance_cm, current_temp, etc.) that are being constantly updated by the sensor functions in the main loop.
- The sizeof(msg) argument ensures the formatted string never exceeds the 350-byte limit of the array.

5. Transmission and Interrupt Re-arming

Finally, the microcontroller must send this massive formatted string back to the user and prepare for the next command.

In your code:

```

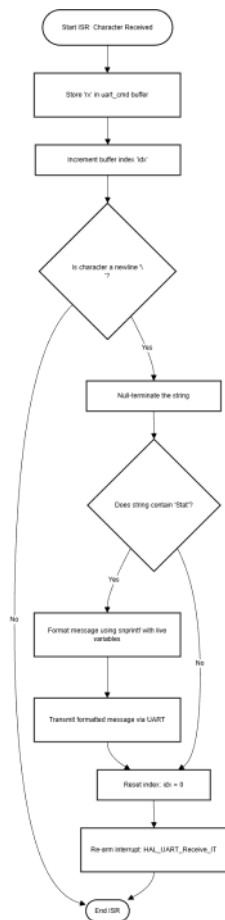
    HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), 100);
    idx = 0;
}
HAL_UART_Receive_IT(&huart2, &rx, 1);

```

- **HAL_UART_Transmit:** This sends the newly constructed msg array out over the UART TX pin. It is given a timeout of 100ms to complete the transmission.
- **Resetting state:** The buffer index (idx) is reset to 0 so the next typed command will start at the beginning of the buffer.
- **Re-arming:** Because HAL_UART_Receive_IT only triggers once per call, it must be called again at the very end of the callback to tell the hardware, "Start listening for the *next* character."

Stat Command Flowchart

Monday, 16 March 2026 21:39



In embedded systems, communicating via UART (like reading user typing from a serial terminal) can be incredibly slow compared to the CPU's speed. If the CPU had to stop and wait for a user to type a full command, it would miss real-time events—like ultrasonic echoes or button presses.

To solve this, your code uses an **Interrupt-Driven** approach. Here is exactly how that process flows:

1. The Interrupt Trigger (Start of ISR)

The main while(1) loop does not actively check the UART. Instead, the hardware has been configured (via HAL_UART_Receive_IT) to listen in the background. The moment a single character (e.g., the letter 'S') arrives at the microcontroller's RX pin, the hardware fires an interrupt.

- The CPU pauses whatever it was doing in the main loop.
- It immediately jumps into the HAL_UART_RxCpltCallback function (the Interrupt Service Routine, or ISR).

2. Buffering the Data

Because commands consist of multiple characters (S-t-a-t), the microcontroller needs to store them as they arrive.

- The code takes the newly received character rx and places it into an array called uart_cmd.
- It uses an index variable idx to keep track of where to put the character, and then immediately increments idx so the *next* character will go into the next slot.

3. The Delimiter Check

The microcontroller needs a way to know when the user has finished typing. In your code, this is the newline character (\n), which is sent when the user presses "Enter".

- **If the character is NOT \n:** The microcontroller assumes the command is still being typed. It skips the rest of the parsing logic, jumps to the end of the ISR to re-arm the interrupt, and goes back to the main loop.
- **If the character IS \n:** The microcontroller knows the command is complete and proceeds to process the buffer.

4. Parsing and Execution (The "Stat" Command)

Once the newline is detected, the code must interpret what the user typed:

- **Null-Termination:** It first adds a \0 (null character) to the end of the uart_cmd array. This transforms the raw array of characters into a standard C-string, which is required for string manipulation functions.
- **Command Checking:** It uses strstr(uart_cmd, "Stat") to check if the word "Stat" exists anywhere in the buffer.
- **Generating the Response:** If "Stat" is found, it uses sprintf to build a massive text block. It pulls the most recent values from your volatile global variables (distance_cm, current_temp, proximity_alarm, etc.) and formats them into a readable string.
- **Transmission:** It uses HAL_UART_Transmit to send this newly formatted status report back to the user over the TX pin.

5. Reset and Re-arm

Before exiting the ISR, the system must reset itself for the next command:

- **Reset idx:** It sets idx = 0. This ensures that the next time the user starts typing, the new characters will overwrite the old command at the beginning of the uart_cmd buffer.
- **Re-arm the Interrupt:** The STM32 HAL library disables the receive interrupt after it triggers once. Your code calls HAL_UART_Receive_IT(&huart2, &rx, 1) at the very end of the function to explicitly tell the hardware, "Start listening for the next character in the background."

Once the interrupt is re-armed, the ISR ends, and the CPU seamlessly resumes whatever it was doing in the main loop right before the interrupt fired.

Demo 2 Software Implementation

Monday, 16 March 2026 20:30

```
/* USER CODE BEGIN Header */
/**
 * *****
 * @file      : main.c
 * @brief     : Main program body
 * *****
 * @attention
 *
 * Copyright (c) 2026 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
 * *****
 */
/* USER CODE END Header */
/* Includes ----- */
#include "main.h"

/* Private includes ----- */
/* USER CODE BEGIN Includes */
#include <string.h>
#include <stdio.h>

/* USER CODE END Includes */

/* Private typedef ----- */
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ----- */
/* USER CODE BEGIN PD */
#define BUTTON_DEBOUNCE_TIME_MS 150
#define TEMP_BUFFER_SIZE 5
#define TRIG_PIN GPIO_PIN_3
#define TRIG_PORT GPIOC
#define TEMP_WINDOW_TIME 1000

/* USER CODE END PD */

/* Private macro ----- */
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ----- */
TIM_HandleTypeDef htim2;
TIM_HandleTypeDef htim3;

UART_HandleTypeDef huart2;

/* USER CODE BEGIN PV */
// LED state tracking
static uint8_t led_d2_state = 1; // D2 starts ON (continuously)
uint8_t led_d3_state = 1; // D3 starts flashing
```

```

static uint8_t led_d4_state = 1; // D4 starts ON (continuously)
uint8_t led_d5_state = 1; // D5 starts flashing

// Flash timing
uint32_t lastFlashTime = 0;
uint8_t flashToggle = 0;

// Debounce trackers for each specific button
uint32_t LastTick_S1_PA0 = 0; // S1
uint32_t LastTick_S2_PB8 = 0; // S2
uint32_t LastTick_S3_PA7 = 0; // S3
uint32_t LastTick_S4_PB6 = 0; // S4
uint32_t LastTick_S5_PB9 = 0; // S5

// Temperature sensor tracking
volatile uint32_t pulse_count = 0;
volatile uint32_t last_capture_time = 0;
volatile float current_temp = -50.0f;
volatile uint8_t temp_ready = 0;

// Temperature sensor calibration
float temp_buffer[TEMP_BUFFER_SIZE] = {0};
uint8_t temp_index = 0;
uint8_t buffer_full = 0;

//Variables for the proximity sensor
volatile uint32_t echo_start = 0;
volatile uint32_t echo_end = 0;
volatile uint8_t echo_captured = 0;
volatile float distance_cm = 0.0f;

// For the UART @Stat Command
uint8_t rx;
char uart_cmd[32];
uint8_t idx = 0;

// For alarm coding
volatile uint8_t proximity_alarm = 0;
volatile uint8_t temperature_alarm = 0;
volatile uint8_t alarm_check_enabled = 0; // alarms disabled at startup

/* USER CODE END PV */

/* Private function prototypes -----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_USART2_UART_Init(void);
static void MX_TIM2_Init(void);
static void MX_TIM3_Init(void);
/* USER CODE BEGIN PFP */
float CalibrateTemperature(float new_temp);

/* USER CODE END PFP */

/* Private user code -----*/
/* USER CODE BEGIN 0 */
void SendStudent_Number(void)
{
    HAL_Delay(350);
    char uart_buffer[20];
    const char studentNumber[] = "*24989142#\n";
    sprintf(uart_buffer, "%s", studentNumber);
    HAL_UART_Transmit(&huart2, (uint8_t*)studentNumber, sizeof(studentNumber)-1,
500);
}

```

```

}

void UpdateLEDs(void)
{
    uint32_t now = HAL_GetTick();
    if((now - lastFlashTime) >= 500)
    {
        lastFlashTime = now;
        flashToggle ^= 1;
    }

    // If alarms enabled, LEDs default ON
    if(alarm_check_enabled)
    {
        // D2
        if(proximity_alarm)
            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, flashToggle ? GPIO_PIN_SET :
GPIO_PIN_RESET);
        else
            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_RESET);

        // D3
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, GPIO_PIN_RESET);

        // D4
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_RESET);

        // D5
        if(temperature_alarm)
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_4, flashToggle ? GPIO_PIN_SET :
GPIO_PIN_RESET);
        else
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_4, GPIO_PIN_RESET);
    }
    else
    {
        // Normal manual LED control
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, led_d2_state ? GPIO_PIN_SET :
GPIO_PIN_RESET);
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, led_d3_state ? GPIO_PIN_SET :
GPIO_PIN_RESET);
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, led_d4_state ? GPIO_PIN_SET :
GPIO_PIN_RESET);
        HAL_GPIO_WritePin(GPIOC, GPIO_PIN_4, led_d5_state ? GPIO_PIN_SET :
GPIO_PIN_RESET);
    }
}

void S1_Button(void)
{
    static uint8_t state = 0;
    static uint32_t press_time = 0;

    uint8_t current = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0);
    if (current == 1 && state == 0)
    {
        // Button just pressed
        press_time = HAL_GetTick();
        state = 1;
    }
    if (current == 0 && state == 1)
    {
        // Button released
        uint32_t duration = HAL_GetTick() - press_time;
    }
}

```

```

        if (duration > 50)    // must be held 100ms
        {
            led_d2_state = !led_d2_state;
            alarm_check_enabled = 0;
            proximity_alarm = 0;
        }
        state = 0;
    }
}

void S2_Button(void)
{
    static uint8_t state = 0;
    static uint32_t press_time = 0;

    uint8_t current = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_8);

    if (current == 1 && state == 0)
    {
        // Button just pressed
        press_time = HAL_GetTick();
        state = 1;
    }
    if (current == 0 && state == 1)
    {
        // Button released
        uint32_t duration = HAL_GetTick() - press_time;
        if (duration > 50)    // must be held 100ms
        {
            led_d3_state = !led_d3_state;
            alarm_check_enabled = 0;
        }
        state = 0;
    }
}

void S3_Button(void)
{
    static uint8_t state = 0;
    static uint32_t press_time = 0;
    uint8_t current = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_7);
    if(current == 1 && state == 0)
    {
        press_time = HAL_GetTick();
        state = 1;
    }
    if(current == 0 && state == 1)
    {
        if(HAL_GetTick() - press_time > 50)
        {
            // Enable alarm system
            alarm_check_enabled = 1;

            // Turn OFF all LEDs
            led_d2_state = 0;
            led_d3_state = 0;
            led_d4_state = 0;
            led_d5_state = 0;
        }
        state = 0;
    }
}

void S4_Button(void)

```

```

{
    static uint8_t state = 0;
    static uint32_t press_time = 0;

    uint8_t current = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_6);

    if (current == 1 && state == 0)
    {
        // Button just pressed
        press_time = HAL_GetTick();
        state = 1;
    }
    if (current == 0 && state == 1)
    {
        // Button released
        uint32_t duration = HAL_GetTick() - press_time;
        if (duration > 50) // must be held 100ms
        {
            led_d4_state = !led_d4_state;
            alarm_check_enabled = 0;
        }
        state = 0;
    }
}

void S5_Button(void)
{
    static uint8_t state = 0;
    static uint32_t press_time = 0;

    uint8_t current = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_9);

    if (current == 1 && state == 0)
    {
        // Button just pressed
        press_time = HAL_GetTick();
        state = 1;
    }
    if (current == 0 && state == 1)
    {
        // Button released
        uint32_t duration = HAL_GetTick() - press_time;
        if (duration > 50) // must be held 100ms
        {
            led_d5_state = !led_d5_state;
            alarm_check_enabled = 0;
            temperature_alarm = 0;
        }
        state = 0;
    }
}

void UpdateTemperature(void)
{
    static uint32_t last_time = 0;
    if(HAL_GetTick() - last_time >= TEMP_WINDOW_TIME)
    {
        last_time = HAL_GetTick();
        __disable_irq();
        uint32_t PC = pulse_count;
        pulse_count = 0;
        __enable_irq();
        if(PC > 10)
        {

```

```

        float temperature = ((float)PC / 4096.0f * 256.0f) - 50.0f;
        current_temp = CalibrateTemperature(temperature);
    }

    temp_ready = 1;

    if(alarm_check_enabled)
    {
        if(current_temp > 30.0f)
            temperature_alarm = 1;
        else
            temperature_alarm = 0;
    }
    else
    {
        temperature_alarm = 0;
    }
}

void Trigger_Ultrasonic(void)
{
    HAL_GPIO_WritePin(TRIG_PORT, TRIG_PIN, GPIO_PIN_SET);
    HAL_Delay(1); // guarantees >10us pulse
    HAL_GPIO_WritePin(TRIG_PORT, TRIG_PIN, GPIO_PIN_RESET);
}

/* USER CODE END 0 */

/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{
    /* USER CODE BEGIN 1 */

    /* USER CODE END 1 */

    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    HAL_Init();

    /* USER CODE BEGIN Init */

    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* USER CODE BEGIN SysInit */

    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_USART2_UART_Init();
    MX_TIM2_Init();
    MX_TIM3_Init();
    /* USER CODE BEGIN 2 */
    SendStudent_Number();

```



```

HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_SET); // D2 ON
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, GPIO_PIN_SET); // D3 starts ON
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_SET); // D4 ON
HAL_GPIO_WritePin(GPIOC, GPIO_PIN_4, GPIO_PIN_SET); // D5 starts ON

flashToggle = 1;
lastFlashTime = HAL_GetTick();

// Initialising Timers
HAL_TIM_IC_Start_IT(&htim3, TIM_CHANNEL_2);
HAL_TIM_IC_Start_IT(&htim2, TIM_CHANNEL_2);

// The stat command
HAL_UART_Receive_IT(&huart2,&rx,1);

static uint32_t last_ultra = 0;

// Initialising alarm conditions
alarm_check_enabled = 0;
proximity_alarm = 0;
temperature_alarm = 0;

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    UpdateLEDs();
    S1_Button();
    S2_Button();
    S3_Button();
    S4_Button();
    S5_Button();
    UpdateTemperature();

    if(HAL_GetTick() - last_ultra > 100)
    {
        last_ultra = HAL_GetTick();
        Trigger_Ultrasonic();
    }
}
/* USER CODE END 3 */
}

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /** Configure the main internal regulator output voltage
    */
    __HAL_RCC_PWR_CLK_ENABLE();
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);

```

```

/** Initializes the RCC Oscillators according to the specified parameters
 * in the RCC_OscInitTypeDef structure.
 */
RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
RCC_OscInitStruct.HSIState = RCC_HSI_ON;
RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
RCC_OscInitStruct.PLL.PLLM = 16;
RCC_OscInitStruct.PLL.PLLN = 336;
RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV4;
RCC_OscInitStruct.PLL.PLLQ = 4;
if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
{
    Error_Handler();
}

/** Initializes the CPU, AHB and APB buses clocks
 */
RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
{
    Error_Handler();
}
}

/**
 * @brief TIM2 Initialization Function
 * @param None
 * @retval None
 */
static void MX_TIM2_Init(void)
{
    /* USER CODE BEGIN TIM2_Init 0 */

    /* USER CODE END TIM2_Init 0 */

    TIM_MasterConfigTypeDef sMasterConfig = {0};
    TIM_IC_InitTypeDef sConfigIC = {0};

    /* USER CODE BEGIN TIM2_Init 1 */

    /* USER CODE END TIM2_Init 1 */
    htim2.Instance = TIM2;
    htim2.Init.Prescaler = 83;
    htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim2.Init.Period = 4294967295;
    htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
    if (HAL_TIM_IC_Init(&htim2) != HAL_OK)
    {
        Error_Handler();
    }
    sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
    sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
    if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) != HAL_OK)

```

```

{
    Error_Handler();
}
sConfigIC.ICPolarity = TIM_INPUTCHANNELPOLARITY_RISING;
sConfigIC.ICSelection = TIM_ICSELECTION_DIRECTTI;
sConfigIC.ICPrescaler = TIM_ICPSC_DIV1;
sConfigIC.ICFilter = 0;
if (HAL_TIM_IC_ConfigChannel(&htim2, &sConfigIC, TIM_CHANNEL_2) != HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN TIM2_Init 2 */

/* USER CODE END TIM2_Init 2 */

}

/**
 * @brief TIM3 Initialization Function
 * @param None
 * @retval None
 */
static void MX_TIM3_Init(void)
{
    /* USER CODE BEGIN TIM3_Init 0 */

    /* USER CODE END TIM3_Init 0 */

    TIM_MasterConfigTypeDef sMasterConfig = {0};
    TIM_IC_InitTypeDef sConfigIC = {0};

    /* USER CODE BEGIN TIM3_Init 1 */

    /* USER CODE END TIM3_Init 1 */
    htim3.Instance = TIM3;
    htim3.Init.Prescaler = 83;
    htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim3.Init.Period = 65535;
    htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    htim3.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
    if (HAL_TIM_IC_Init(&htim3) != HAL_OK)
    {
        Error_Handler();
    }
    sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
    sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
    if (HAL_TIMEx_MasterConfigSynchronization(&htim3, &sMasterConfig) != HAL_OK)
    {
        Error_Handler();
    }
    sConfigIC.ICPolarity = TIM_INPUTCHANNELPOLARITY_RISING;
    sConfigIC.ICSelection = TIM_ICSELECTION_DIRECTTI;
    sConfigIC.ICPrescaler = TIM_ICPSC_DIV1;
    sConfigIC.ICFilter = 0;
    if (HAL_TIM_IC_ConfigChannel(&htim3, &sConfigIC, TIM_CHANNEL_2) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN TIM3_Init 2 */

    /* USER CODE END TIM3_Init 2 */

}

```

```

/**
 * @brief USART2 Initialization Function
 * @param None
 * @retval None
 */
static void MX_USART2_UART_Init(void)
{
    /* USER CODE BEGIN USART2_Init 0 */

    /* USER CODE END USART2_Init 0 */

    /* USER CODE BEGIN USART2_Init 1 */

    /* USER CODE END USART2_Init 1 */
    huart2.Instance = USART2;
    huart2.Init.BaudRate = 57600;
    huart2.Init.WordLength = UART_WORDLENGTH_9B;
    huart2.Init.StopBits = UART_STOPBITS_1;
    huart2.Init.Parity = UART_PARITY_EVEN;
    huart2.Init.Mode = UART_MODE_TX_RX;
    huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
    huart2.Init.OverSampling = UART_OVERSAMPLING_16;
    if (HAL_UART_Init(&huart2) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN USART2_Init 2 */

    /* USER CODE END USART2_Init 2 */

}

/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    /* USER CODE BEGIN MX_GPIO_Init_1 */

    /* USER CODE END MX_GPIO_Init_1 */

    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOC_CLK_ENABLE();
    __HAL_RCC_GPIOH_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_3|GPIO_PIN_4, GPIO_PIN_RESET);

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOA, LD2_Pin|GPIO_PIN_10, GPIO_PIN_RESET);

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4|GPIO_PIN_5, GPIO_PIN_RESET);

    /*Configure GPIO pin : B1_Pin */
    GPIO_InitStruct.Pin = B1_Pin;

```

```

GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(B1_GPIO_Port, &GPIO_InitStruct);

/*Configure GPIO pins : PC3 PC4 */
GPIO_InitStruct.Pin = GPIO_PIN_3|GPIO_PIN_4;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

/*Configure GPIO pins : PA0 PA7 */
GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_7;
GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
GPIO_InitStruct.Pull = GPIO_PULLDOWN;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

/*Configure GPIO pins : LD2_Pin PA10 */
GPIO_InitStruct.Pin = LD2_Pin|GPIO_PIN_10;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

/*Configure GPIO pins : PB4 PB5 */
GPIO_InitStruct.Pin = GPIO_PIN_4|GPIO_PIN_5;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

/*Configure GPIO pins : PB6 PB8 PB9 */
GPIO_InitStruct.Pin = GPIO_PIN_6|GPIO_PIN_8|GPIO_PIN_9;
GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
GPIO_InitStruct.Pull = GPIO_PULLDOWN;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

/* EXTI interrupt init*/
HAL_NVIC_SetPriority(EXTI0_IRQn, 0, 0);
HAL_NVIC_EnableIRQ(EXTI0_IRQn);

HAL_NVIC_SetPriority(EXTI9_5_IRQn, 0, 0);
HAL_NVIC_EnableIRQ(EXTI9_5_IRQn);

HAL_NVIC_SetPriority(EXTI15_10_IRQn, 0, 0);
HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);

/* USER CODE BEGIN MX_GPIO_Init_2 */

/* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */
float CalibrateTemperature(float new_temp)
{
    temp_buffer[temp_index++] = new_temp;

    if (temp_index >= TEMP_BUFFER_SIZE)
    {
        temp_index = 0;
        buffer_full = 1;
    }
    if (!buffer_full) return new_temp;
}

```

```

// Copy buffer
float temp_copy[5];
memcpy(temp_copy, temp_buffer, sizeof(temp_buffer));

// Sort (simple bubble sort)
for (int i = 0; i < 4; i++)
    for (int j = i + 1; j < 5; j++)
        if (temp_copy[i] > temp_copy[j])
        {
            float t = temp_copy[i];
            temp_copy[i] = temp_copy[j];
            temp_copy[j] = t;
        }

// Average middle 3 values
return (temp_copy[1] + temp_copy[2] + temp_copy[3]) / 3.0f;
}

void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    // TEMPERATURE SENSOR (TIM3)
    // TEMPERATURE SENSOR (TIM3)
    if (htim->Instance == TIM3 && htim->Channel == HAL_TIM_ACTIVE_CHANNEL_2)
    {
        uint32_t current_time = HAL_GetTick();
        // Detect start of new pulse burst
        if((current_time - last_capture_time) > 4)
        {
            // new burst starting
            pulse_count = 0;
        }
        pulse_count++;
        last_capture_time = current_time; // count pulse
        // remember pulse time
    }
    // PROXIMITY SENSOR (HC-SR04)
    if ((htim->Instance == TIM2) && (htim->Channel == HAL_TIM_ACTIVE_CHANNEL_2))
    {
        uint32_t capture = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_2);

        if(echo_captured == 0)
        {
            echo_start = capture;
            echo_captured = 1;
            __HAL_TIM_SET_CAPTUREPOLARITY(htim, TIM_CHANNEL_2,
TIM_INPUTCHANNELPOLARITY_FALLING);
        }
        else
        {
            echo_end = capture;
            echo_captured = 0;
            __HAL_TIM_SET_CAPTUREPOLARITY(htim, TIM_CHANNEL_2,
TIM_INPUTCHANNELPOLARITY_RISING);
            uint32_t diff;
            if(echo_end >= echo_start)
                diff = echo_end - echo_start;
            else
                diff = (0xFFFFFFFF - echo_start + echo_end + 1);

            float time_us = diff;
            distance_cm = time_us * 0.0343f / 2.0f;
            proximity_alarm = (alarm_check_enabled && distance_cm < 10.0f);
        }
    }
}

```

```

    }
}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if(huart->Instance == USART2)
    {
        if(idx < sizeof(uart_cmd)-1)
        {
            uart_cmd[idx++] = rx;
        }
        if(rx == '\n')
        {
            uart_cmd[idx] = '\0';
            if(strstr(uart_cmd,"Stat"))
            {
                char msg[350];
                snprintf(msg, sizeof(msg),
"@2026/02/26 22:12:42\n"
"Distance: %.1f cm\n"
"Temperature: %.1f C\n"
"Light: 0000 lux\n"
"X accel: 0.00 g\n"
"Y accel: 0.00 g\n"
"Z accel: 0.00 g\n"
"Unsafe driving: 0\n"
"Impact detected: 0\n"
"Low-Light warning: 0\n"
"Proximity warning: %d\n"
"High Temperature: %d\n"
"GPS lat: 0.00000\n"
"GPS long: 0.00000\n"
"&\n",
                                distance_cm, current_temp, proximity_alarm,
temperature_alarm);

                HAL_UART_Transmit(&huart2,(uint8_t*)msg,strlen(msg),100);
            }
            idx = 0;
        }
        HAL_UART_Receive_IT(&huart2,&rx,1);
    }
}

/* USER CODE END 4 */

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1)
    {
    }
    /* USER CODE END Error_Handler_Debug */
}

#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number
 * where the assert_param error has occurred.
 * @param file: pointer to the source file name

```

```

* @param line: assert_param error line source number
* @retval None
*/
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and line number,
       ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
    /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */

```


Button Operation (System State Operations)

Monday, 16 March 2026 22:16



Button Operations (System State Control)

Your buttons do much more than just turn lights on and off; they act as the inputs for a **State Machine** that governs the entire behavior of the embedded system.

- **Step 1: Detecting a Valid Press**
 - **What it does:** As explained in the debounce flowchart, the system waits until a button has been pressed, held for >50ms, and completely released before it executes any of the following steps.
- **Step 2: The "Master Arm" Sequence (Button S3)**
 - **What it does:** When S3 is validated, the code sets alarm_check_enabled = 1. It also actively sets all four manual LED tracking variables (led_d2_state through led_d5_state) to 0.
 - **Why:** Setting the flag tells the UpdateLEDs, UpdateTemperature, and Ultrasonic functions to start monitoring safety thresholds. Forcing the LED variables to 0 is a clever design choice: it ensures that when the user eventually disarms the system, all the LEDs will default to OFF, rather than remembering whatever random pattern they were in before the system was armed.
- **Step 3: "Disarm and Toggle" (Buttons S2 & S4)**
 - **What it does:** Pressing S2 or S4 flips their respective LED state variables (led_d3_state or led_d4_state). Crucially, it also forces alarm_check_enabled = 0.
 - **Why:** If the system is currently armed and the user presses a manual lighting button, the system assumes the user wants manual control. By instantly clearing the alarm flag, the system drops out of monitoring mode and hands control back to the user without requiring them to press S3 again.
- **Step 4: "Acknowledge and Clear" (Buttons S1 & S5)**
 - **What it does:** S1 and S5 behave exactly like S2 and S4 (toggling their LED and disarming the system), but they add one extra step: S1 forces proximity_alarm = 0, and S5 forces temperature_alarm = 0.
 - **Why:** These specific buttons are tied to the specific LEDs (D2 and D5) that flash during an alarm. If the proximity alarm is flashing, pressing S1 acts as an "Acknowledge" button. It turns off the active alarm flag and disarms the system, ensuring the warning is completely cleared from the system's memory.

Non-Blocking Button Debounce Flowchart

Monday, 16 March 2026 21:51

To understand the **Non-Blocking Button Debounce Flowchart**, we need to break it down into two separate problems that your code is brilliantly solving: **Debouncing** and **Non-Blocking execution**.

1. What is "Debounce"?

When you press a physical, mechanical button, the metal contacts inside don't just perfectly connect once. Microscopically, they smash together and literally "bounce" off each other for a few milliseconds before settling down.

To a fast microcontroller, this bouncing looks like you pressed and released the button 10 or 20 times in a row. If you mapped a button to turn an LED on and off, the LED would flicker wildly and end up in a random state.

Debouncing is the software logic used to filter out that noise. It essentially says: *"I see the pin changed state, but I'm going to wait a few milliseconds to make sure it stays that way before I count it as a real press."*

2. What is "Non-Blocking"?

The easiest way to debounce a button is to use a "blocking" delay, like this:

```
if (button_pressed) {  
    HAL_Delay(50); // Wait 50ms for the bouncing to stop  
    if (button_still_pressed) {  
        // Turn on LED  
    }  
}
```

However, `HAL_Delay()` **blocks** the CPU. The entire microcontroller freezes for 50 milliseconds. It can't update LEDs, it can't trigger the ultrasonic sensor, and it can't check other buttons. In a real-time system, freezing the CPU is a terrible idea.

Your code uses a **Non-Blocking** approach. It uses `HAL_GetTick()` (the system's internal millisecond clock) to record *timestamps* instead of pausing the code.

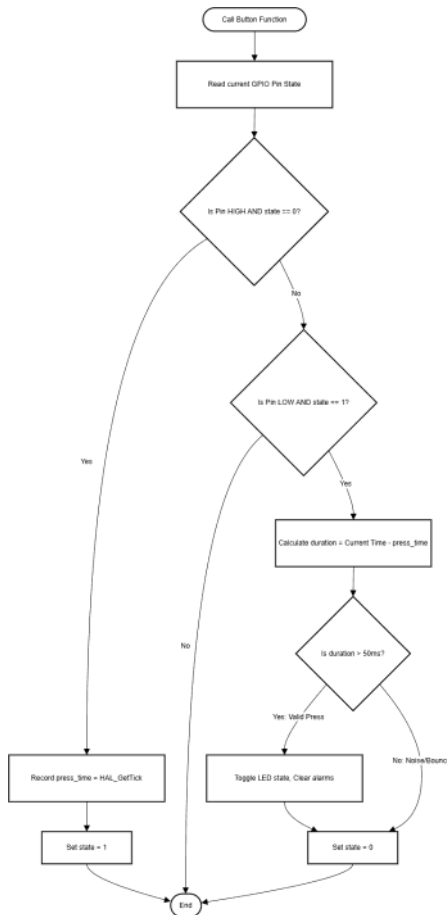
How Your Flowchart Combines Them

Instead of waiting, your code uses a "State Machine." It remembers what the button was doing the last time the loop ran, checks the current time, and moves on immediately.

Here is the logic walking through the flowchart:

- **State 0 (Idle):** The code sees the button just went HIGH (it was pressed). Instead of pausing the CPU, it simply looks at the clock, writes down the time (`press_time = HAL_GetTick()`), changes its internal memory to `state = 1`, and **instantly moves on** to the rest of the main loop.
- **State 1 (Waiting for Release):** The main loop comes back around hundreds of times. The button might be physically bouncing, but the code ignores the bouncing because `state` is already 1.
- **The Release:** Eventually, the user lets go of the button, and the pin goes LOW. The code sees this and looks at the clock again. It subtracts the original `press_time` from the current time.
- **The Decision (The Debounce):** * If the duration was **less than 50ms**, the code assumes it was just a tiny electrical noise spike or a bounce, ignores it, and resets to State 0.
 - If the duration was **greater than 50ms**, the code knows a human finger definitively held the button down and released it. It executes your action (toggling the LED state) and resets to State 0.

Because there are no `HAL_Delay()` functions, your main loop spins thousands of times per second, managing 5 different buttons simultaneously without ever missing a beat!



In embedded systems, communicating over UART can be very slow compared to the blazing speed of the CPU. If the CPU had to sit and wait in a while loop for the user to type out a full command character by character, it would completely freeze the rest of the system (missing button presses or sensor readings). To prevent this, your code uses **interrupts**. Here is the step-by-step logic of how the flowchart and code handle this:

1. The Background Listener (Interrupt Trigger)

In your main while(1) loop, there is no code actively checking the UART. Instead, the hardware is configured to listen in the background. When the user types a single character (e.g., the letter 'S'), that byte arrives at the RX pin. The hardware instantly fires an interrupt, which pauses the main loop and forces the CPU to jump into the HAL_UART_RxCpltCallback function (the Interrupt Service Routine, or ISR).

2. Storing the Data (Buffering)

Because a command consists of multiple letters, the microcontroller must collect them one by one as they arrive.

- Inside the ISR, the new character (rx) is saved into an array named `uart_cmd`.
- The code uses an index variable (idx) to place the character in the correct slot, and then increments idx so the next character goes into the next available slot.
- It also includes a safety check (`idx < sizeof(uart_cmd)-1`) to ensure a user typing a massive string doesn't overflow the buffer and crash the system.

3. Detecting the End of the Command (Delimiter)

The system needs to know when the user has finished typing. It does this by checking for a delimiter, which in this case is the newline character (`\n` or "Enter").

- **If the character is NOT `\n`:** The microcontroller assumes the user is still typing. It skips the parsing steps, jumps to the end of the ISR to re-arm the interrupt, and immediately resumes the main loop.
- **If the character IS `\n`:** The microcontroller knows the full command has been entered and moves to the parsing stage.

4. Parsing the Command

Once the newline is detected, the code interprets the stored characters:

- **Null-Termination:** It adds a `\0` (null terminator) to the end of the `uart_cmd` array. This is required in C to turn a raw array of characters into a readable string.
- **Keyword Search:** It uses the standard C function `strstr(uart_cmd, "Stat")` to look for the specific command "Stat" inside the buffer.

5. Formatting and Transmitting the Response

If the "Stat" command is recognized, the microcontroller gathers the requested data and sends it back.

- **Formatting:** It uses `snprintf` to create a large text block. This function takes your live volatile variables (like `distance_cm`, `current_temp`, `proximity_alarm`) and injects them directly into the formatted string text.
- **Transmitting:** It calls `HAL_UART_Transmit` to send this newly built status report out through the TX pin and back to the user's terminal.

6. Resetting for the Next Command

Before leaving the ISR, the system must clean up so it is ready for whatever the user types next:

- **Reset the Index:** It sets `idx = 0`. This ensures the next typed character will start at the very beginning of the `uart_cmd` array, effectively overwriting the old command.
- **Re-arming the Interrupt:** The HAL library automatically disables the receive interrupt after it catches a byte. To keep the cycle going, the code calls `HAL_UART_Receive_IT(&huart2, &rx, 1)` at the very end. This tells the hardware, "Okay, I'm done processing this one; start listening for the next character."

Once the interrupt is re-armed, the ISR ends, and the CPU goes right back to the main loop exactly where it left off.

Sam's Notes

Monday, 16 March 2026 19:21



Scanned Documents

TEST 4 PREP 2025

DEMO 1 Stuff

1. DESIGN OF VOLTAGE REGULATORS

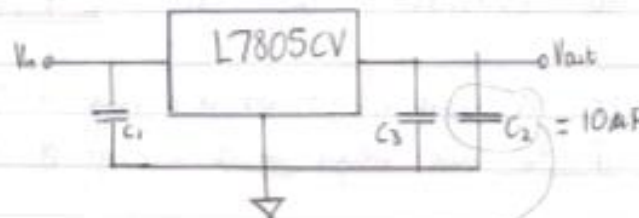
5V Regulator L7805CV



Input Capacitor (C_i) → Stabilizes the input voltage and filters out high frequency noise.

Output Capacitor (C_o) → Improves the transient response and output stability by reducing the output voltage ripple and noise caused by sudden load changes.

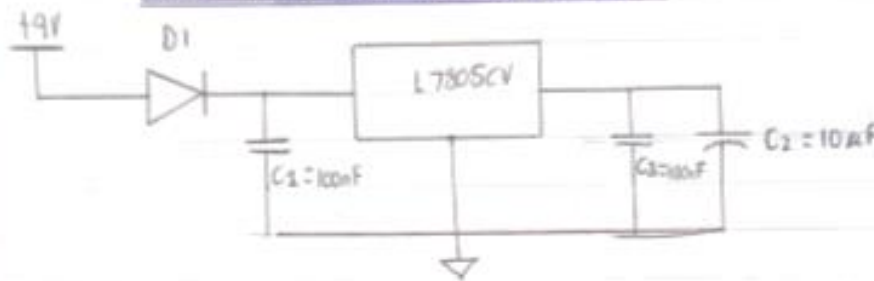
The Role of the Additional Output Capacitor C_2 on the 5V regulator circuit



Absorbs voltage spikes resulting from a switching load quickly, thus protecting the circuit from unwanted transients.

→ It is worth noting that the capacitor C_2 could have been left out because the L7805CV has its own internal frequency compensation built into its design.

The Role of the Diode D1



- * The role of Diode D1 is to ensure the direction of current flow is toward the regulator, thereby protecting the bench power supply in the event of malfunctions on the circuit board.

→ 1N4007 diode was used, but any low-voltage drop diode could be used.

CALCULATIONS ASSOCIATED WITH L7805CV REGULATOR

N.B all properties of the Regulator we used should be that in Table 11 pg 15

Q1 Unintended Change in the input voltage on Bench Power Supply Vs change in Output Voltage of the Regulator :-

"Given :- Temperature remains constant at 25°C"

have to consider the " ΔV_o " Line regulation how

- * If the input voltages are in a range of 7 to 25V $\rightarrow \Delta V_o = 100mV$
- * If the input voltages are in a range of 8 to 12V $\rightarrow \Delta V_o = 50mV$

Example :- You powered the device using a ^{9V} power supply in labs. When plugging something in, you accidentally changed the output to 10V while the device was powered. What is the most you'd expect the output voltage of L7805 to change?

$$9V \xrightarrow{10} 10V \quad \Delta V_o = 100mV$$

Q2 Sudden change in LOAD current Vs the change in the Output Voltage ΔV_o

"Given :- Temperature remains constant at 25°C "

HAVE TO CONSIDER the " ΔV_o " LOAD REGULATION ROW

* If the load current ranges from 5mA to $1.5\text{A} \rightarrow \Delta V_o = 100\text{mV}$

* If the load current ranges from 250mA to $750\text{mA} \rightarrow \Delta V_o = 50\text{mV}$

Q3 Change in temperature Vs Output voltage of Regulator

"Given : that the load current is $I_o = 5\text{mA}$ "

HAVE TO GO TO $\Delta V_o / \Delta T$ column :-

$$\frac{\Delta V_o}{\Delta T} = -1.1 \text{ mV}/^\circ\text{C}$$

Example :- What is the new output voltage if the temperature were to increase to 72°C .

$$\frac{V_f - V_i}{T_f - T_i} = -1.1$$

$$V_f = -1.1 \frac{\text{mV}}{^\circ\text{C}} [T_f - T_i] + V_i$$

$$V_f = \frac{-1.1}{1000} [72 - 25] + 5$$

$$V_f = 5.0517\text{V}$$

The start current doesn't really specify the initial output voltage, but know that from our project the desired output is 5V

Q4 Calculating the minimum temperature at which the regulator operates at to decide whether heat sink is needed or not?

* For $T_O = 220$ package that L7805CV that we use is in, the Thermal resistance junction - ambient constant is 50°C (from table 2 pg 5)

$$\theta_{ja} = 50^\circ\text{C}/\text{W}$$

$$\text{Power} = V_{\text{across Regulator}} \times I_{\text{max. desired by Load}}$$

$$\text{Power} = [V_{\text{in}} - V_{\text{out}}] \times I_{\text{max Load}}$$

* Say for instance, 9V is supplied from bench power supply and only 5V comes out of the regulator and the load current is 110mA

$$\text{Power} = [9 - 5][110 \times 10^{-3}] = 0.44 \text{ W}$$

$$T_{\text{Jmax}} = \text{Power} \times \theta_j + T_{\text{amb}}$$

Let's say you are given an ambient temperature of $T_{\text{amb}} = 25^\circ\text{C}$

$$\text{Then } T_{\text{Jmax}} = [0.44 \text{ W} \times 50^\circ\text{C/W}] + 25^\circ\text{C} = 47^\circ\text{C}$$

* From table 1 (pg 5), the operating junction temperature range is "0 to 150°C ", and since for our design we got 47°C which falls within the range, no heat sink needed.

Q5 What is the absolute maximum input voltage that should be input to the L7805CV.

* The maximum input voltage for the L7805CV is dependant on the desired output voltage or the voltage at which the input voltage is to be regulated down to.

$$\star \text{ For } V_{\text{out}} = 5 \text{ to } 18 \text{ V} \Rightarrow \text{MAX INPUT V} = 35 \text{ V}$$

$$\star \text{ For } V_{\text{out}} = 20, 24 \text{ V} \Rightarrow \text{MAX INPUT V} = 40 \text{ V}$$

Q6 CALCULATING MINIMUM AND MAXIMUM POWER DISSIPATED BY L7805 REGULATOR

$$P_{\text{delivered by Regulator}} = V_{\text{drop across REGULATOR}} \cdot I_{\text{load}}$$

$$P_{\text{delivered by Regulator}} = (V_{\text{in}} - V_{\text{out Regulated}}) \cdot \frac{V_{\text{to Load}}}{R_{\text{load}}}$$

Given the input voltage range: $[V_{\text{min}}; V_{\text{max}}]$ and Load Resistance (R_{load}), the min and max power dissipated by the device are calculated as follows

$$\star P_{\text{min}} = [V_{\text{min}} - V_{\text{out Regulated}}] \cdot \frac{V_{\text{out Regulated}}}{R_{\text{load}}}$$

$$\star P_{\text{max}} = [V_{\text{max}} - V_{\text{out Regulated}}] \cdot \frac{V_{\text{out Regulated}}}{R_{\text{load}}}$$

Q7 CALCULATING POWER DISSIPATED IN THE REGULATOR

$$P_{\text{Dissipated in the Regulator}} = (V_{\text{applied}} - V_{\text{out Regulated}}) (I_{\text{load}} + I_{\text{d}})$$

→ Power dissipated in the regulator is equal to voltage drop across it multiplied by total current (Current delivered to Load + Current used internally in the Regulator (I_{d}))

* For L7805CV, $I_{\text{d}} = 8 \text{ mA}$ (Table 11 p15)

e.g. $V_{\text{in}} = 17.5 \text{ V}$, $R_{\text{load}} = 39 \text{ ohms}$ and $I_{\text{d}} = 8 \text{ mA}$

$$P_{\text{Dissipated in Regulator}} = (17.5 - 5) \left(\frac{5}{39} + \frac{8}{1000} \right) = 1.7026 \text{ W}$$

~~UART~~

The use of Jumper J30 ??

→ Allows test station to supply the student board with a nominal 9V.

The use of Jumper J31 ??

→ Allows 9V supply to regulator.

The use of Fuse F1 ??

→ Allows supply of 5V to STM32 board.

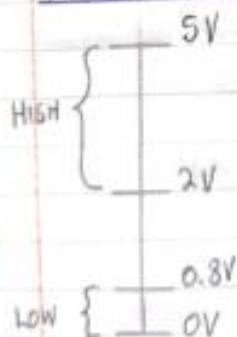
UART

"The UART shall operate at a baud rate of 57600 bps, with 8 data bits, 1 parity bit, and one stop bit, using TTL signal levels. The parity used will be even parity"

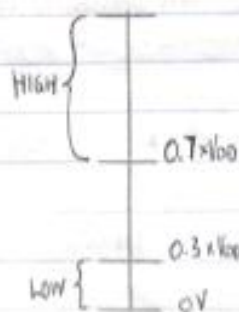
ACCEPTABLE TTL SIGNAL LEVELS DEPENDING ON WHETHER THE SIGNAL IS AN INPUT OR OUTPUT

Acceptable TTL Gate

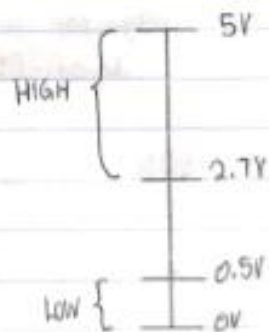
INPUT SIGNAL LEVELS



"NON-BOOT"



OUTPUT SIGNAL LEVELS

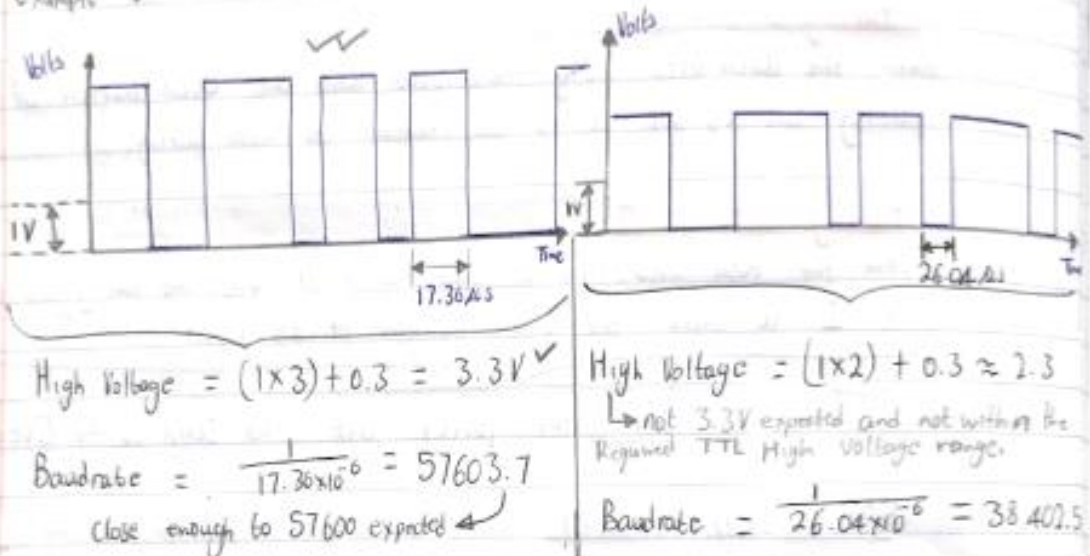


You gotta check if the High of the given signal is within the range of 2.7V to 5V or not? ; Check if the Low is within the range of 0V to 0.5V

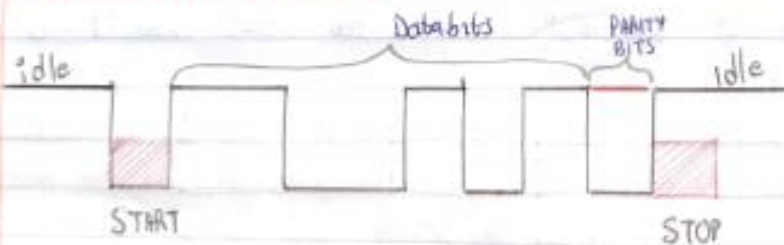
* FOR OUR SYSTEM THE HIGH SHOULD BE 3.3V and Low should be 0V.

* Baud Rate should be 57600 bps.

Example :-



UART FRAME FORMAT



Data bits → CAN BE 7 OR 8 depending on the specifications.

↳ MB! On the UART FRAME FORMAT THE LEAST SIGNIFICANT BIT IS SENT FIRST, meaning when attempting to find the sent character you need to REARRANGE to MSB → LSB

PARITY BIT → An extra bit added to detect errors.

WHEN IS THE PARITY BIT EQUAL TO 0 and WHEN IS IT EQUAL TO 1 ??

Parity bit = 0

When the data bits being transmitted have an even number of 1s, the parity bit is set to 0 to maintain the even parity.

Parity bit = 1

When the data bits have an odd number of 1s, the parity bit is set to 1 to make the total number of 1s even

This is when the parity bit is said to be Even!

FOR ODD PARITY

* Number of 1s in the databits even $\Rightarrow$ PARITY = 1

* Number of 1s in the databits odd $\Rightarrow$ PARITY = 0

WHAT DOES IT MEAN TO BE EXPECTING A CERTAIN PARITY TYPE FROM A GIVEN SIGNAL BUT THE NUMBER OF 1s IN THE DATABITS AND THE PARITY BIT DON'T AGREE WITH THE EXPECTATION?

C.g. Expecting Even parity

Number of 1s in databits = Even

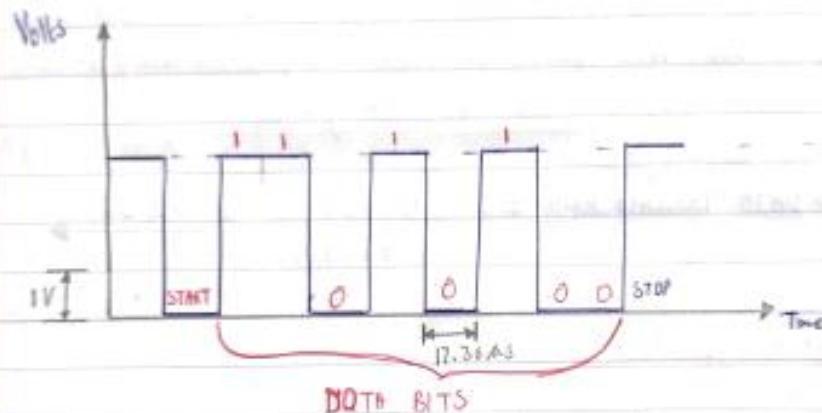
Parity bit = 1

→ This implies corruption of data during transmission.

WHAT DOES IT IMPLIES TO BE EXPECTING TO HAVE A CERTAIN PARITY BIT, BUT IT IS NOT AVAILABLE ON THE GIVEN ^{SIGNAL'S} UART FRAME FORMAT??

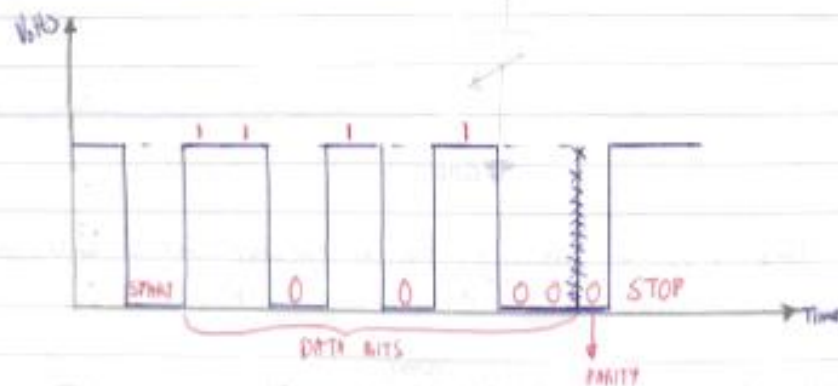
Example: UART SPECS :- 57600 bps ; 8 data bits ; 1 parity bit and 1 Stop bit

Given Signal :-



! (*) The above shows an invalid signal according to the expected specs, since the parity bit is expected but not there! and this means communication will fail, and not character or data will be transmitted

The CORRECT SIGNAL WOULD BE :-



Transmitted character : MSB → LSB
00101011 = +

CALCULATING MAXIMUM DATA TRANSFER RATE ON CHANNEL (Byte/Second)

Given the UART Specs :-

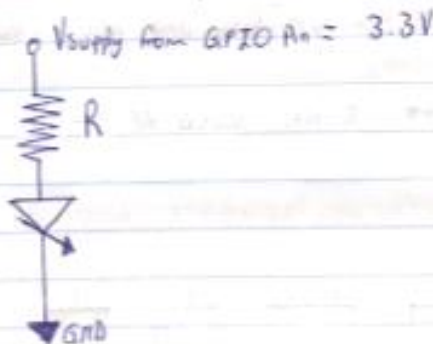
$$\text{MAX DATA TRANSFER RATE} = \frac{\text{BAUD RATE}}{\sum \text{of all bits}} = \frac{\text{BAUD RATE}}{\text{\# of data bits} + \text{START bit} + \text{STOP bit} + \text{PARITY bit}}$$

Example :- UART port operates at 57600 baud ; 8-bit data ; 1-start bit ; 1-stop bit and even parity bit

$$\text{MAX DATA TRANSFER RATE} = \frac{57600}{8+1+1+1} = 5236$$

DEBUG LEDs

* The LED needs current limit through a series resistor, so as to not damage the processor output pins.



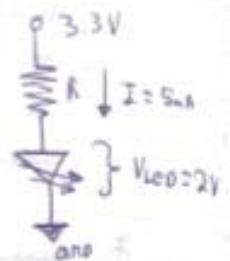
* This requires knowing the voltage drop across LED, which from the datasheet is 1.9V to 2.6V and TYPICAL = 2V.

* KNOWING THAT "OPERATIONAL ^{OUTPUT} CURRENT" OF GPIO pin goes up to 20mA.
→ NOT ABSOLUTE MAX! remember we never design for absolute MAX for anything.

EXAMPLE: CHOOSE current to be 5mA and calculate R:-

$$\frac{V_{supply} - V_{LED}}{I} = R$$

$$\frac{3.3 - 2}{5 \times 10^{-3}} = 260 \Omega$$



STANDARD E12

$$\text{For } < 10 \Omega \Rightarrow \begin{bmatrix} 1.0 ; 1.2 ; 1.5 ; 1.8 ; 2.2 ; 2.7 ; 3.3 ; 3.9 ; \\ 4.7 ; 5.6 ; 6.8 ; 8.2 \end{bmatrix}$$

$$\text{For } 10 \Omega \leq R < 100 \Omega \Rightarrow \begin{bmatrix} 10 ; 12 ; 15 ; 18 ; 22 ; 27 ; 33 ; 39 ; 47 ; \\ 56 ; 68 ; 82 \end{bmatrix}$$

$$\text{For } R \geq 1000 \Omega \Rightarrow \begin{bmatrix} 1000 ; 1200 ; 1500 ; 1800 ; 2200 ; 2700 ; 3300 ; 3900 ; 4700 ; \\ 5600 ; 6800 ; 8200 \end{bmatrix}$$

$\Rightarrow$ CHOOSE THE CLOSEST !!

DEMO 2 STUFF

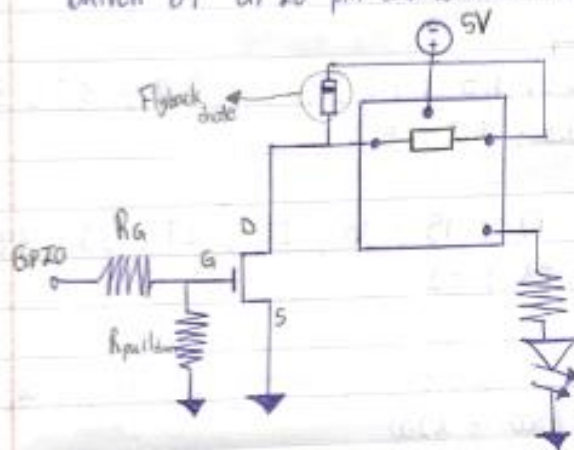
RELAY - MOSFET CIRCUIT

WHY RELAY CANNOT BE DRIVEN DIRECTLY FROM GPIO PINS?

1. The J107 series relay requires a minimum voltage (pick up voltage) of 3.75V in order for it to transition from one state to the other (ON/OFF), which is greater than 3.3V maximum voltage that can be supplied by GPIO pin.

2. The coil requires a current of 72.5 mA for it to be able to generate a magnetic field that will induce the movement of the switch to turn the load ON/OFF, But the GPIO pin can supply a maximum of 20 mA, and out of this 20 mA already some current is being used up by LED D2.

* Since the Relay Requires higher Voltage and current than what the GPIO pin can supply; A MOSFET IS USED SUCH THAT ~~IT IS~~ IT IS DRIVEN BY GPIO pin and connected to Relay to complete its circuit.



2N7000 MOSFET

* The mosfet is a voltage driven device which required a very small current at the gate for it to turn on.

→ Since MOSFET is a voltage driven device, the circuit design has to be decided based on the **GATE THRESHOLD VOLTAGE** :-

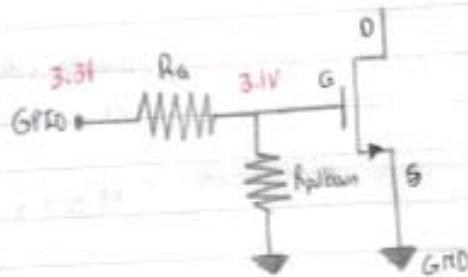
$$V_{as(th)} = 0.8V \text{ to } 3V$$

↑	GUARANTEED ON STATE	3V
	NO MAN'S LAND ~ NO GUARANTEED OPERATION	0.8V
↓	GUARANTEED OFF STATE	

PURPOSE OF SMALL SERIES RESISTOR R_g ON THE MOSFET CIRCUIT AND ITS VALUE CALCULATION

$R_g \rightarrow$ prevents in-rush current to the mosfet.

CALCULATING R_g :-



Since there's no specific requirement for gate current, despite the fact that ~~it should be very small~~ IT SHOULD BE VERY SMALL
 + CHOOSE $I = 1\text{mA}$
 and turn on voltage of $V_{on} = 3.1\text{V}$

$$R_g = \frac{3.3 - 3.1\text{V}}{1\text{mA}} = 200\Omega$$

PURPOSE OF THE LARGE PULL-DOWN RESISTOR AND CALCULATING ITS VALUE

The large pull-down resistor ensures that the mosfet gate is firmly pulled to 0V (ground) when the control signal is inactive (low), and this helps prevent accidental turn-on due to leakage currents or floating gate conditions.

CALCULATING $R_{pull-down}$:-

$R_{pull-down}$ is typically calculated based on the series gate resistor (R_g).

The rule of thumb is :-

$$R_{pull-down} \geq 10 \times R_g$$

To ensure a large enough value is used, chose to make it 10 times the minimum

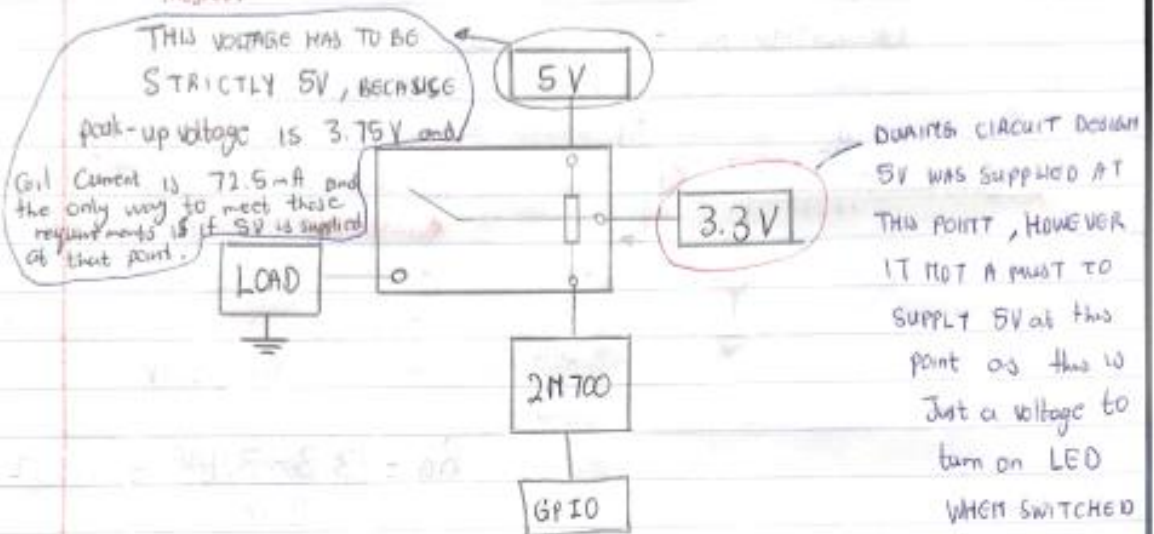
$$R_{pull-down} = 10 \times 10 \times R_g$$

$$R_{pull-down} = 10 \times 10 \times 200$$

$$R_{pull-down} = 2000\Omega$$

PURPOSE OF FLYBACK DIODE

→ The flyback diode prevents the high voltage spikes that gets generated when the relay switches OFF; it also creates path and direction to which the energy that the coil stores can be dissipated to avoid damaging mosfet.

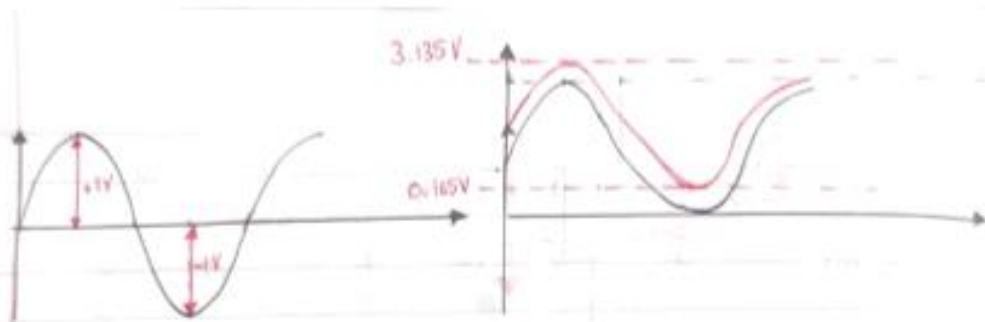


ADC MEASUREMENTS

WHY IS THE OP-AMP CIRCUIT NEEDED?

* The given signal is Alternating signal and includes negative values. However, ADC cannot measure negative voltages and therefore need to scale the values to be in the range of 0-3.3V. Also need to make sure that the full ADC resolution is used to ensure accuracy of the results. Hence, adding offset and multiplying by the gain ~~to~~ through OP-AMP.





Even though the ADC input voltage range is 0-3.3V but design for absolute maximum values is avoided. This is ~~because~~ to avoid "Clipping" due to component tolerances introducing variations that pushes the values beyond the expected range hence damaging the GPIO pins.

Design range is $[0.165V ; 3.135V]$ instead of $[0V ; 3.3V]$

CALCULATING GAIN AND OFFSET

Lower peak Amplitude : Desired Lower Peak Amplitude
 $-1V \Rightarrow 0.165V$

Higher Peak Amplitude : Desired Higher Peak Amplitude
 $1V \Rightarrow 3.135V$

$$\begin{array}{l} V_{in} \quad V_{out} \\ [-1V ; 0.165V] \\ [1V ; 3.135V] \end{array}$$

$$\begin{array}{l} V_{out} = \text{Gain} \cdot V_{in} + V_{offset} \\ V_{out} = m \cdot V_{in} + b \end{array}$$

$$0.165 = m(-1) + b$$

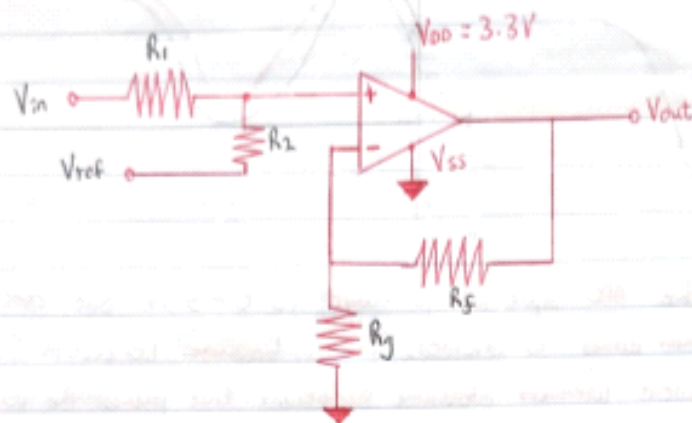
$$b = 0.165 + m \quad \dots \text{--- ①}$$

$$3.135V = m(1) + b \quad \dots \text{--- ②}$$

$$3.135 = m + 0.165 + m$$

$$m = 1.485$$

$$b = 1.65$$



WHAT SHOULD THE SUPPLY VOLTAGE V_{DD} BE? and WHY?

The datasheet specifies the range of input supply voltage to be 2.7V to 6.0V, HOWEVER in our case V_{DD} needs to be strictly set to 3.3V. This is to ensure that should there be connection errors leading to saturation of the output, the ADC maximum input voltage is not exceeded! This is clearly explained by the equations used to find the output range at different conditions depending on the value of V_{DD} .

~~Maximum output voltage swing~~ ~~When output is at VDD or VSS~~

→ From DC Characteristics Table - Output Section

MAXIMUM OUTPUT VOLTAGE SWING

CONDITIONS

- * $V_{OL} = V_{SS} + 15\text{mV}$; $V_{OH} = V_{DD} - 20\text{mV}$ @ $R_L = 25\text{k}\Omega$ to V_L ; output drive = 0.5V
- * $V_{OL} = V_{SS} + 45\text{mV}$; $V_{OH} = V_{DD} - 60\text{mV}$ @ $R_L = 5\text{k}\Omega$ to V_L ; output drive = 0.5V

LINEAR OUTPUT VOLTAGE SWING

- * $V_{OUT_LOW} = V_{SS} + 100\text{mV}$; $V_{OUT(HIGH)} = V_{DD} - 100\text{mV}$ @ $R_L = 25\text{k}\Omega$ to V_L ; $A_{OL} \geq 100\text{dB}$
- * $V_{OUT_LOW} = V_{SS} + 100\text{mV}$; $V_{OUT(HIGH)} = V_{DD} - 100\text{mV}$ @ $R_L = 5\text{k}\Omega$ to V_L ; $A_{OL} \geq 95\text{dB}$

EXAMPLE:- Say for instance Question says it is MAXIMUM OUTPUT VOLTAGE SWING and the conditions are $R_L = 5\text{k}\Omega$ to V_L ; Output drive = 0.5V :-

If $V_{DD} > 3.3V$ let's say $5V$, then the voltage range going to ADC is :-

$$\left[\overset{V_{LH}}{V_{SS} + 45mV} ; \overset{V_{HH}}{V_{DD} - 60mV} \right]$$

$$\left[0.045V ; 4.94V \right]$$

$4.94V$ is way to greater than $3.3V$ (rather $3.6V$ as specified by MCU datasheet) which is the maximum voltage that ADC peripheral of the microcontroller can handle. This then means you are running the risk of damaging the microcontroller's ADC peripheral.

WHAT SHOULD V_{ref} BE? and WHY??

⇒ V_{ref} was also set to $3.3V$, and this is because V_{ref} acts as a virtual ground and represents the offset voltage at which the sine wave should be symmetric to allow ^{usable} dynamic range. This V_{ref} must be stable and noise-free, as any noise or fluctuation would cause the entire signal to shift unpredictably, leading to inaccurate ADC readings or even clipping.

⊛ Therefore, $3.3V$ from our circuit board has already been well regulated, stabilized and incorporated with decoupling capacitors, hence its use for V_{ref} .

Finding VALUES FOR R_1 ; R_2 ; R_F and R_g ??

* R_1 and R_F should be given, e.g. assume $R_1 = R_F = 10k\Omega$

* $R_2 = \frac{V_{ref} \times R_1 \times m}{b}$ where $m = \text{Gain}$; $b = V_{offset}$

* $R_g = \frac{R_2 \times R_F}{m \times (R_1 + R_2) - R_2}$

ADC, Setup and Sampling Data Management

TIM TRIGGERED ADC IN DMA MODE WAS USED → Allows time regulated ^{continuous} sampling and storing of data to memory without CPU intervention.

DMA Advantages

- Allows ADC to directly transfer the data to memory without CPU intervention. [Peripheral to memory]
- Allows continuous high-speed ADC conversions without CPU intervention, ideal for scanning multiple ADC channels since it can automatically transfer data from multiple channels to memory.

WHAT SHOULD SAMPLING TIME BE SET TO? and WHY?

⇒ Needs to be set to highest available sampling time, to allow transient effects on the signal to settle.

CALCULATING SAMPLING FREQUENCY AND NUMBER OF SAMPLES??

Typical theorem:- $f_{\text{sampling}} \geq 2 \times f_{\text{signal}}$

e.g. Say $f_{\text{signal}} = 50 \text{ Hz}$
 $f_{\text{sampling}} = 200 \times 2 \times 50 \text{ Hz} = 20 \text{ kHz}$

$$\text{Number of Samples} = \frac{f_{\text{sampling}}}{f_{\text{signal}}}$$

$$\text{Number of Samples} = \frac{20000}{50} = 400 \text{ samples}$$

Buffer Size $\geq 2 \times$ Number of Samples → ensures enough cycles of signal's wave are fed to ensure stability.

CALCULATING COUNTER PERIOD (ARR) AKA TIMER PERIOD

$$ARR + 1 = \frac{\text{Internal Clock Frequency}}{(\text{Prescaler} + 1) \cdot f_{\text{sampling}}}$$

CALCULATING CONVERSION TIME (TIME IT TAKES TO COMPLETE ONE FULL SEQUENCE OF BOTH SIGNALS)

$$T_{\text{conv}} = \frac{\text{Sampling Time} + 12.5}{\text{ADC CLOCK}}$$

$$\therefore \text{Total } T_{\text{conv}} = 2 \times T_{\text{conv}}$$

ADC VALUE $\leftrightarrow$ VOLTAGE

$$\text{ADC VALUE} = \frac{V_{\text{in}} \times \text{ADC Resolution}}{V_{\text{ref}}}$$

$$\text{ADC VALUE} = \frac{V_{\text{in}}}{V_{\text{ref}}} \times (2^n - 1)$$

CALCULATING PEAK-TO-PEAK VALUES

WHY NOT USING $2 \times V_{\text{max}} = V_{\text{pp}}$, even though it is correct mathematically??

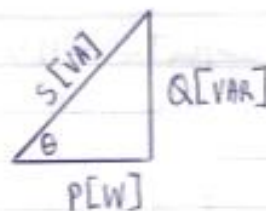
$\Rightarrow V_{\text{pp}} = 2 \times V_{\text{max}}$ assumes that the wave is perfectly centered at 0V ~~zero~~ or whatever offset (i.e. no asymmetry). But in reality the waveform may not ^{be ideal} (due to gain error; offset; load, or temperature drift) therefore even small offsets cause large errors when computing peak-to-peak without subtracting min from max.

THEREFORE CORRECT METHOD :- $V_{\text{pp}} = V_{\text{max}} - V_{\text{min}}$

DEMO 3 STUFF

WHY MEASURE THE PHASE DIFFERENCE?

- Voltage and current in an electrical system don't peak at exactly the same time whenever there's any reactance in the load (even though their input parameters are same i.e frequency and V_{pp}). Therefore, knowing phase angle or phase difference allows us to differentiate on how much power is actually delivered "Real power" and how much is just "Reactive power"



Apparent Power : $S = V_{rms} \cdot I_{rms}$

Real Power : $P = S \cos(\theta)$

Reactive Power : $Q = S \sin(\theta)$

WHY DO WE NEED POWER FACTOR? [AKA EFFICIENCY : FRACTION OF USEFUL POWER] ~ Percentage or Ratio of Real Power!!

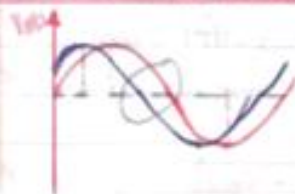
- Power factor tells us the fraction of the Apparent Power (S) that is actually doing useful work :-

$$PF = \frac{P_{real}}{P_{apparent}} = \frac{S \cos \theta}{S} = \cos \theta$$

$PF = 1 \rightarrow$ All supplied energy is doing useful work.

$PF < 1 \rightarrow$ Some energy is lost to sustaining magnetic / electric fields.

CALCULATING PHASE ANGLE

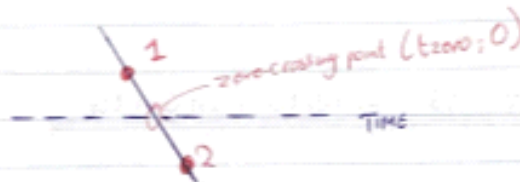


$$PF = \frac{P_{real}}{P_{apparent}} = \frac{P_{real}}{\sqrt{(P_{real})^2 + (P_{reactive})^2}} = \frac{S \cos \theta}{\sqrt{(S \cos \theta)^2 + (S \sin \theta)^2}} = \cos \theta$$

* To get phase angle, it is necessary to find the time difference between two points of the given waves. WHY ZERO CROSSING POINTS HIGHLY RECOMMENDED OVER THE PEAKS?

⇒ At sine wave's peaks the slope is zero, any small voltage noise or ADC quantization error will translate into a large uncertainty in the inferred peak time, thus inaccurate calculations, whereas at zero crossings waveform's slope is maximal, so the voltage uncertainty produces much smaller timing error and this is ideal for simple linear interpolation.

FINDING ZERO CROSSING TIME :-



point 1 : $(t_1 ; y_1)$
 zero-crossing point : $(t_{zero} ; 0)$
 point 2 : $(t_2 ; y_2)$

interpolation : $\frac{t_{zero} - t_1}{t_2 - t_1} = \frac{0 - y_1}{y_2 - y_1}$

$$t_{zero} = \frac{-y_1}{y_2 - y_1} (t_2 - t_1) + t_1$$

$$t_{zero} = t_1 - \frac{y_1}{y_2 - y_1} (t_2 - t_1)$$

CHANGE IN TIME (Δt) ??

$$\Delta t = t_{zero, voltage} - t_{zero, current}$$

CALCULATING PHASE ANGLE ??

$$\begin{aligned}\phi &= \omega \cdot \Delta t \\ \phi &= 2\pi f_{\text{signal}} \cdot \Delta t\end{aligned}$$

in Normal way: $\phi = \theta_v - \theta_i$

WHY PHASE ANGLE HAD TO BE RESTRICTED TO $\pm 90^\circ$??

$\Rightarrow$ Phase angle has to be restricted to $\pm 90^\circ$ to ensure that the real power stays positive all the time. Otherwise, if it becomes negative it will seem as if the load is capable of generating the power whereas it only consumes it.

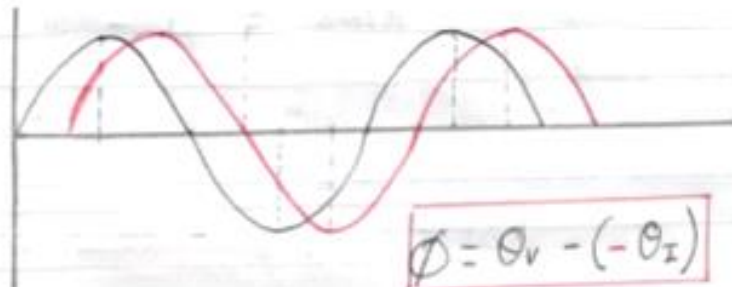
PHASE ANGLE VS LEADING / LAGGING

CASE 1: CURRENT LEADING VOLTAGE [PHASE ANGLE < 0]



$$\phi = \theta_v - |\theta_i|$$

CASE 2: VOLTAGE LEADS CURRENT [PHASE ANGLE > 0]



$$\phi = \theta_v - (-\theta_i)$$

OLED

* Uses I2C interface :-

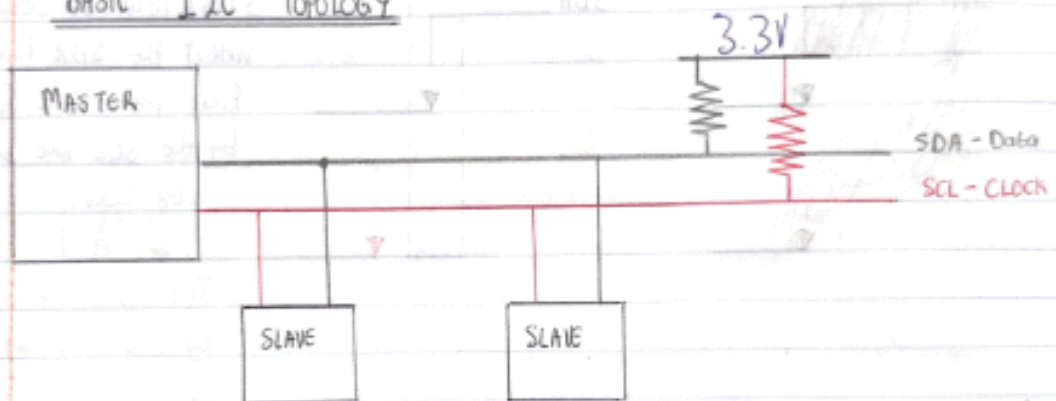
SDA → Serial Data

↳ Carries the data bits, one at a time, bi-directionally.

SCL → Serial Clock

↳ Carries the clock pulses, always generated by the master.

BASIC I2C TOPOLOGY



* In normal "IDLE" state of the I2C (i.e. when no communication occurring)
⇒ SDA and SCL lines are pulled High, by the pull-up resistors shown on the topology above.

WHY ARE PULL-UP RESISTORS REQUIRED FOR CLOCK AND DATA LINES OF I2C PORT

⇒ The devices (slaves) connected to I2C port pull the SDA and SCL lines low, but they cannot drive it high. Therefore, the pull-up resistors ensure that the bus lines always return to the logic high level when idle (no communication).
↳ Without pull-up resistors, the SDA and SCL lines would just float, leading to undefined logic levels and communications.

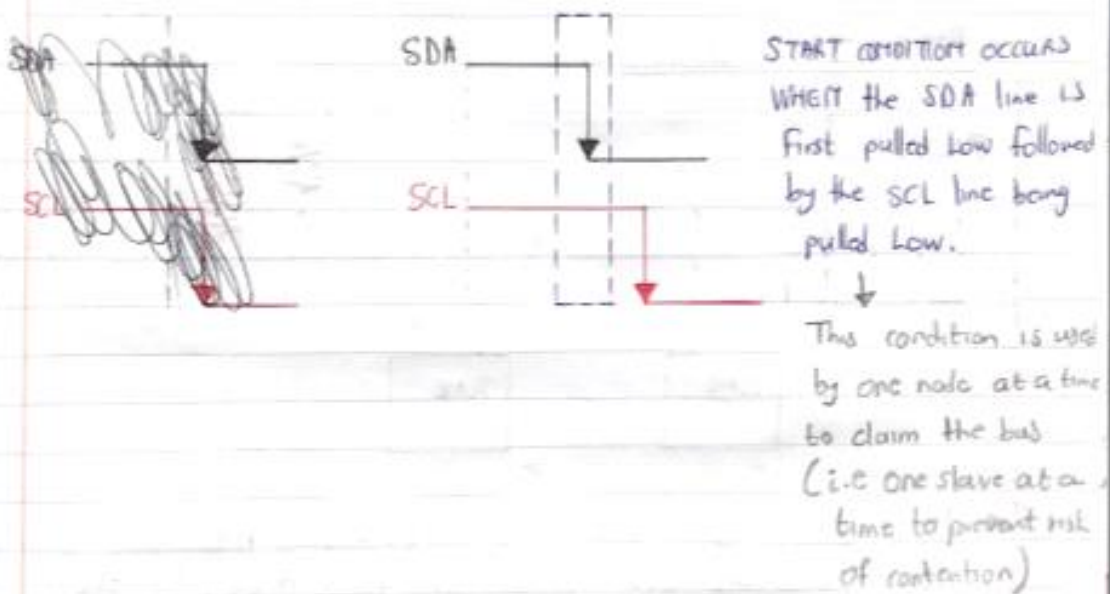
* The range of these pull-up resistors : $[1k\Omega \text{ to } 10k\Omega]$

* Higher pull-up resistance → LOWER BUS SPEED, and lower resistance → FAST SPEED

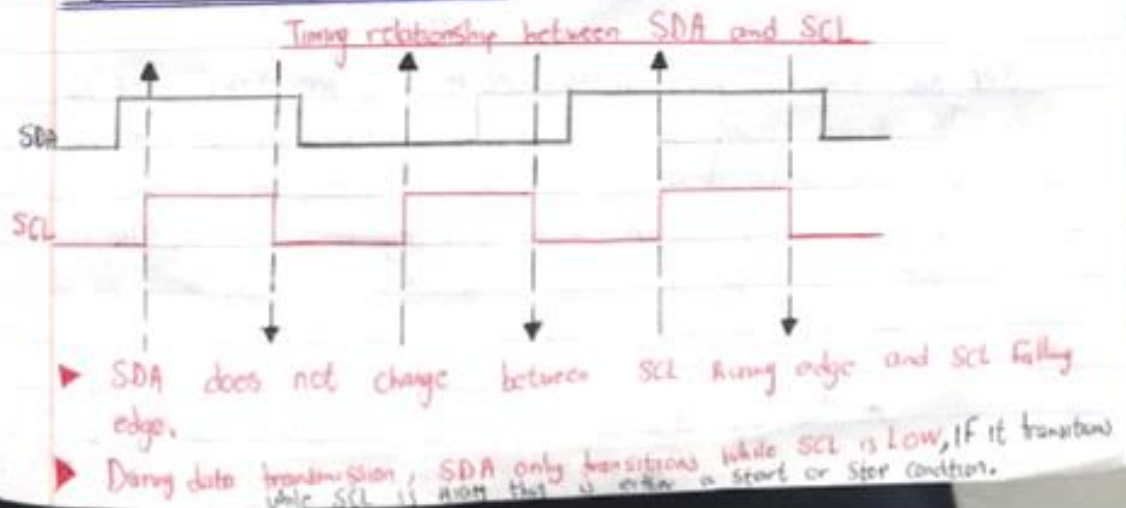
I2C MODES OR SPEEDS

I2C MODE	SPEED
Standard Mode	100 kHz
Fast Mode	400 kHz
Fast Mode plus	1000 kHz

START CONDITION FOR I2C



DURING DATA TRANSMISSION



Git Cheat Sheet

Monday, 16 March 2026 20:29



git-cheat-sh
eet-educa...

GitHub

GIT CHEAT SHEET

Git is the free and open source distributed version control system that's responsible for everything GitHub related that happens locally on your computer. This cheat sheet features the most important and commonly used Git commands for easy reference.

INSTALLATION & GUIs

With platform specific installers for Git, GitHub also provides the ease of staying up-to-date with the latest releases of the command line tool while providing a graphical user interface for day-to-day interaction, review, and repository synchronization.

GitHub for Windows

<https://windows.github.com>

GitHub for Mac

<https://mac.github.com>

For Linux and Solaris platforms, the latest release is available on the official Git web site.

Git for All Platforms

<http://git-scm.com>

SETUP

Configuring user information used across all local repositories

```
git config --global user.name "[firstname lastname]"
```

set a name that is identifiable for credit when review version history

```
git config --global user.email "[valid-email]"
```

set an email address that will be associated with each history marker

```
git config --global color.ui auto
```

set automatic command line coloring for Git for easy reviewing

SETUP & INIT

Configuring user information, initializing and cloning repositories

```
git init
```

initialize an existing directory as a Git repository

```
git clone [url]
```

retrieve an entire repository from a hosted location via URL

STAGE & SNAPSHOT

Working with snapshots and the Git staging area

```
git status
```

show modified files in working directory, staged for your next commit

```
git add [file]
```

add a file as it looks now to your next commit (stage)

```
git reset [file]
```

unstage a file while retaining the changes in working directory

```
git diff
```

diff of what is changed but not staged

```
git diff --staged
```

diff of what is staged but not yet committed

```
git commit -m "[descriptive message]"
```

commit your staged content as a new commit snapshot

BRANCH & MERGE

Isolating work in branches, changing context, and integrating changes

```
git branch
```

list your branches. a* will appear next to the currently active branch

```
git branch [branch-name]
```

create a new branch at the current commit

```
git checkout
```

switch to another branch and check it out into your working directory

```
git merge [branch]
```

merge the specified branch's history into the current one

```
git log
```

show all commits in the current branch's history



INSPECT & COMPARE

Examining logs, diffs and object information

git log

show the commit history for the currently active branch

git log branchB...branchA

show the commits on branchA that are not on branchB

git log --follow [file]

show the commits that changed file, even across renames

git diff branchB...branchA

show the diff of what is in branchA that is not in branchB

git show [SHA]

show any object in Git in human-readable format

TRACKING PATH CHANGES

Versioning file removes and path changes

git rm [file]

delete the file from project and stage the removal for commit

git mv [existing-path] [new-path]

change an existing file path and stage the move

git log --stat -M

show all commit logs with indication of any paths that moved

IGNORING PATTERNS

Preventing unintentional staging or committing of files

```
logs/  
*.notes  
pattern*/
```

Save a file with desired patterns as `.gitignore` with either direct string matches or wildcard globs.

git config --global core.excludesfile [file]

system wide ignore pattern for all local repositories

SHARE & UPDATE

Retrieving updates from another repository and updating local repos

git remote add [alias] [url]

add a git URL as an alias

git fetch [alias]

fetch down all the branches from that Git remote

git merge [alias]/[branch]

merge a remote branch into your current branch to bring it up to date

git push [alias] [branch]

Transmit local branch commits to the remote repository branch

git pull

fetch and merge any commits from the tracking remote branch

REWRITE HISTORY

Rewriting branches, updating commits and clearing history

git rebase [branch]

apply any commits of current branch ahead of specified one

git reset --hard [commit]

clear staging area, rewrite working tree from specified commit

TEMPORARY COMMITS

Temporarily store modified, tracked files in order to change branches

git stash

Save modified and staged changes

git stash list

list stack-order of stashed file changes

git stash pop

write working from top of stash stack

git stash drop

discard the changes from top of stash stack

GitHub Education

Teach and learn better, together. GitHub is free for students and teachers. Discounts available for other educational uses.

✉ education@github.com

🌐 education.github.com